KH정보교육원

요구사항 확인

목차

Part 1. 현행 시스템 분석하기

Chapter 1. 웹 애플리케이션 개발 환경 분석 ····· 4

Chapter 2. 웹 애플리케이션 개발 환경 구축 ····· 10

Chapter 3. 형상관리도구 git 사용 ····· 18

Part 2. 요구사항 확인하기

Chapter 4. 웹 애플리케이션 기능 요구사항 분석 ····· 45

Chapter 5. 요구사항 정의 명세서 작성 ····· 47

Chapter 6. 기능별 USE CASE 다이어그램 작성 ····· 49

Part 3. 분석모델 확인하기

Chapter 7. 기능별 상세 명세서 작성 ····· 59

Chapter 8. 요구 기능별 애플리케이션 설계 ····· 66

Part 1. 현행 시스템 분석하기

Chapter 1. 웹 애플리케이션 개발 환경 분석

웹 서버를 개발할 때 요구되는 요소들

웹 생태계의 스펙

- HTML, HTTP(1.1 , HTTP/2)

기본 SDK, 라이브러리/프레임워크 이해와 활용

클라이언트를 위한 API 설계

서버/컴퍼넌트/객체 간의 역할 분담/의존성/통신 방법 설계

저장소 활용

- DBMS 설계
- Cache 적용
  - Global/Local cache 적용범위, 라이프 싸이클, 솔루션 선택
- 파일 저장 정책/솔루션 선택 활용

검색엔진 연동 방식 결정

빌드 도구

- Maven/Gradle

배포 전략

성능 테스트/프로파일링/튜닝

- JVM 레벨의 튜닝 (GC 옵션 등)
  - 웹 서버(Nginx,Tomcat)등의 설정/튜닝
- OS 설정의 주요 값 확인

인접 기술에 대한 이해

- DBMS, Front End 등

서버 개발자에만 해당하지는 않는 항목

- 테스트 코드 작성/리팩토링 기법
- 버전 관리 전략
  - branch 정책 등

팀을 이룬 개발자들의 지식들이 합쳐져서 구현 방식과 정책이 결정됩니다.
프로젝트의 스킬셋을 정할 때에도 보통 구성원 중 적어도 한명은 경험이 있는 기술들을 택합니다. 사용자가 많은 서비스라면 프로그램이 돌아가는 만드는 것만이 목적이 아니기 때문에 기술의 선택에 더 조심스러워 집니다. 운영환경에서의 모니터링과 문제해결까지 팀 구성원이 할 수 있는 기술을 선택합니다.
내부의 소수만 사용하는 관리 도구를 만들 때는 상대적으로 부담 없이 팀원 모두가 처음 쓰는 기술을 적용하기도 합니다.
새로운 기술은 위험성이 적은 서비스에 적용해서 경험을 키운 이후에 큰 서비스에 적용을 합니다.

사용자의 요청량과 저장 용량이 많은 서비스에서는 하나의 저장소만을 쓰지는 않습니다.
네이버의 서비스에서도 MySQL, CUBRID, Redis, Memchaced, HBase, MonoDB, Elasticsearch 등 다양한 저장소를 활용하고 있습니다. 네이버와 라인에서 Arcus, Elasticsearch, Cassandra, Redis, HBase 와 같은 다양한 저장소가 쓰인 사례는 아래 글을 참고하실 수 있습니다.
Storm 과 Elasticsearch Percolator 를 이용한 NELO2 알람 기능 개선
LINE 소셜 네트워크 서비스의 아키텍처 를 참고하실 수 있습니다.
LINE 스토리지, 한달에 수십억 건의 데이터를 Redis 와 HBase 에 저장하다
다양한 저장소가 쓰이는 시대에도 RDB(관계형 데이터베이스)는 여전히 가장 우선시되는 저장소입니다. 그래서 RDB 를 잘 다루는 능력은 백엔드 개발자의 핵심 역량 중 하나입니다. 개발을 하는 도중에도 쿼리의 호출 횟수나 실행 계획이 비효율적이지 않은지 확인하는 습관이 필요합니다. 운영 중에도 느린 쿼리를 모니터링하고 DBA 와 협업하여 성능 개선을 하는 작업을 실무개발자들은 꾸준히 하고 있습니다. ORM 같은 추상화된 프레임워크를 써서 직접 SQL 을 작성하지 않는 경우에도 그런 작업들은 더욱 중요합니다.
대용량 서비스들을 보면 DB 쿼리를 만드는 스타일이 과거와는 달라졌습니다. 과거에는 서버 간의 네트워크 호출 비용을 줄이기 위해 굉장히 많은 테이블을 한번에 조인하는 긴 SQL 을 만드는 경우가 많았습니다. 하지만 요즘은 복잡한 JOIN 은 가급적 피하는 경향이 생겼습니다. 데이터를 조회하는 SQL 이 단순할수록 데이터를 다른 저장소에 캐시하거나 분산해서 저장하기가 쉬워집니다. 대용량을 저장하는 UGC 서비스에서는 RDB 테이블 사이의 JOIN 은 최대한 제약을 하고 어플리케이션 레벨에서 여러 저장소의 연관된 데이터를 조합하기도 합니다.
Stored prodecure 도 가급적 사용하지 않는 경우가 많습니다. DB 안에서 실행되는 Stored procedure 는 급하게 개발된 서비스에서는 많이 사용되었습니다. 네트워크 호출비용이 없어서 성능에 이득이 있고, DB 안에 저장되니 배포절차가 단순했기 때문입니다. 그러나 길게 작성된 Stored prodecure 는 만들었던 사람도 수정하기 힘든 경우가 많습니다. 데이터와 독립적으로 로직을 테스트하기도 어렵습니다. 별다른 배포절차가 없으니 버전관리가 제대로 되지 않는 경우가 많았습니다. 그리고 데이터의 연산에 DB 서버의 CPU 자원을 소모함으로서 서비스가 커가면서 DB 에 병목이 될수 있는 가능성을 더 키울 수 있습니다. 이런 이유로 초기에 Stored procedure 로 개발했던 로직을 어플리케이션 단으로 빼는 작업이 서비스가 성장하는 과정에서 흔하게 일어납니다.
DB 서버 1 대로 트래픽이나 저장량이 감당이 안 될 때, 이떻게 이를 분산할지도 항상 어려운 과제입니다. 성능 향상을 위해서 Local cache, Global cache 를 동원하기도 합니다. 어느 정도 복제지연(Replication replay)이 그다지 민감하지 않은 서비스에서는 쓰기 작업은 Master 노드로, 읽기작업은 복제되는 Master 의 데이터를 복제한 여러 대의 Slave 로 DB 를 구성하기도 합니다. 총 저장되는 용량이 많을 때에는 여러 개의 DB 인스턴스에 이를 나누어서 저장하기도 합니다. 이를 샤딩이라고 부르는데 Sharding Platform 에서 자세한 개념을 참조하실 수 있습니다. Spider, MySQL fabric 와 같은 솔루션으로 샤딩을 할 수 있고 네이버 내부에서는 이런 용도로 자체 개발한 Nbase-T 라는 저장플랫폼을 쓰고 있습니다. 샤딩의 기준이 되는 키가 명확하면 나누어서 저장하기가 상대적으로 쉽습니다. 개인화 서비스의 경우에는 사용자의 ID 별로 샤드키를 잡으면 자연스럽습니다. 그런데 샤드키를 어떤 것으로 잡아야 할지 명확히 확신이 서지 않는 서비스가 있을 수 있고, 중간에 샤드키를 바꾸는 비용은 굉장히 큽니다. 어떤 솔루션을 쓰든 RDB 는 사용량이 늘어났을 때 분산하는 비용이 비쌉니다. 그래서 성장할 가능성이 큰 서비스라면 RDB 의 자원을 아껴서 쓸 필요가 있습니다.

개발툴

개발도구를 잘 활용하는 능력은 생산성과 직결되기에 중요합니다. 그런데 개발도구를 '배워야'하는 개발자보다는 스스로 익힐 수 있고, 적절한 도구를 선택할 수 있는 개발자가 현장에서는 필요합니다. 특정 개발도구를 익혔다는 사실은 단기적으로는 실력이라고 할 수 있습니다. 새로운 도구가 나왔을 때도 적응할 수 있는 학습력/적응력/판단력이 본질이고 이것이 누적되어 실력이 됩니다.

병렬처리

Servlet 기반의 Java 웹서버들은 기본적으로 사용자의 요청을 병렬적으로 처리합니다. 그래서 객체가 멀티스레드에서 공유되는 것인지, 아닌지를 의식하는 일은 중요합니다. 클래스의 멤버변수에는 항상 멀티스레드에서 접근해도 안전한(Thread-safe)한 변수만 두면 된다는 단순한 규칙만으로도 많은 문제를 예방할 수 있습니다. 그런데 Local cache 를 적용할 때는 멀티스레드에서 공유된 객체가 쉽게 눈에 안 띌 수도 있습니다. 그래서 Cache 대상의 객체는 Immutable 하게 유지하는 것이 안전합니다. 직접 Cache 모듈을 만드는 경우 위험한 버그가 생기는 경우가 많습니다. 하나의 메모리 누수를 잡기까지 도 그런 사례입니다. 사용자 요청을 처리하는 스레드 외에도 별도의 스레드 풀로 실행해야 할 작업이 종종 생기기도 합니다. 예를 들면 사용자에게 주는 응답에 영향을 주지는 않지만 실행시간이 길어질 수도 있는 update 구문을 DB 에 실행하는 작업같은 것입니다. 그런 경우 Java 에서는 Executors, ThreadPoolExecutor 에 있는 많은 옵션들이 정교한 제어를 하는데 도움이 됩니다. new Thread()로 직접 스레드를 생성하는 방식은 JDK5 이후로는 권장하지 않습니다.

보안

널리 알려면 XSS, CSRF, SQL Injection 공격에 대해서 대처하는 방법은 모든 개발자가 알고 있어야 합니다. 실무에서 일하다보면 의존하는 라이브러리에서 발견된 보안 취약점에 대처한 경우가 많았습니다. 보통 버전업을 하거나 회사 내에서 자체적인 패치를 하기도 했습니다. 보안 취약점이 자주 발견되는 Struts2 같은 프레임웍들은 신경써서 대처하기 위해 주요 변경을 알리는 메일링 그룹에 가입하기도 했습니다.

테스트

많은 사람이 협업해서 개발하고 지속적으로 개선해나갈 소프트웨어라면 테스트 코드를 작성하는 일은 더욱 중요합니다. 테스트 코드를 작성하는 능력도 백엔드 개발자의 핵심역량 중 하나이기도 합니다. FE 개발이 분업화되고 서비스간의 API 호출이 많아지면서 최근 백엔드 개발자의 업무는 API 서버 개발에 더 집중되고 있습니다. 따라서 최종적으로 UI 와 통합하기 전에 개발한 API 를 스스로 테스트해야 할 필요성이 더 커졌습니다. 오류를 있다는 제보를 다른 개발자로부터 받아서 수정 후 재배포하고 다시 알리는 비용은 스스로 오류를 발견했을 때보다 굉장히 크기 때문입니다. HTTP API 에 대한 테스트는 작성하기도 쉽고 작성했을 때의 이득도 큽니다. 최근 진행 중인 프로젝트에서는 Rest-assured 와 Spring MVC Test Integration 을 이용해서 HTTP API 를 통합 테스트 하고 있습니다. 라이브러리나 개발플랫폼을 개발하는 경우에도 테스트 코드는 중요합니다. 수천 대의 서버에 배포되는 라이브러리에서 사소한 수정을 해서 배포한 적이 있었는데, 최대한 모든 경로를 상상해서 테스트 코드를 작성할 수 밖에 없었습니다. 라이브러리 배포 후 이를 적용한 서비스에서 오류가 발생했을 경우 재배포하는 과정이 비용도 크고 신뢰를 잃게하는 요인이 되기 때문입니다.

'시간이 없어서 테스트를 못 만들었다'는 말은 '나는 테스트 코드를 만드는데 시간이 많이 걸린다'는 말과 동일합니다. 해당 언어에 대한 숙련도가 떨어지는 사람일수록 테스트 코드를 작성하는데 부담을 크게 느끼는 경우도 많이 봤습니다. 능숙해질수록 테스트 작성 시간은 줄어들어 테스트에 투자한 대비 이득이 커집니다.

가끔 테스트를 작성할 때 이 테스트 덕분에 어느 시점에 실제적인 이득이 있을지 [illegible] 생각할 때가 있습니다. 예를 들면 아래와 같습니다.

유지보수 기간의 생산성을 높여주고 새로 프로젝트에 투입될 사람에게도 이득을 주는 테스트

프로젝트 오픈 일정 직전까지의 코드 변경과 버그 발견에 도움을 주는 테스트

오늘 당장 프로그램을 목표한 곳까지 작성하는 일을 더 빨리 마치게 해주는 테스트

테스트 코드를 작성하다 보면 실제로 당장 할 일을 더 마치게 빠르게 해주는 테스트도 만나게 됩니다. 복잡하게 얽힌 프로그램을 개발할 때는 최종적인 UI 를 통해서 수동으로 테스트하기 전에 부분적으로 잘라서 테스트하는 것이 버그를 쉽게, 빨리 잡는데 도움이 됩니다. 예를 들어 일주일 걸려서 만든 프로그램을 마지막 날에 한번에 테스트한다면 디버깅에 훨씬 시간이 많이 걸릴 것입니다. 몇 달 간 진행하는 프로젝트에서 단 한 줄의 테스트 코드도 안 짜는 분이 있다면 당신의 지금 능력으로도 그 방식이 오늘 일을 가장 빨리 마치는 방법이 아닐 것 같다고 말씀드리고 싶습니다. 그리고 테스트를 적극적으로 짜다보면 오늘 하루를 넘어서서 더 큰 이득을 주는 테스트를 만들어서 다른 사람의 생산성에도 긍정적인 영향을 미칠 수 있을 것입니다.

실제로 테스트로 인한 긍정적인 경험을 쌓아가다 보면 더 넓은 범위와 다양한 기법으로 테스트 코드를 작성하는데 동기유발이 됩니다. 제 개인적인 경험을 돌아보면, 점진적으로 테스트 코드를 작성하는 범위를 늘려가는 방식이 도움이 되었습니다. 처음에는 테스트가 쉬운 Utilty 클래스에 대한 테스트부터 작성했습니다. 전에도 간단한 유틸리티에 대한 테스트는 main 메서드 안에서 하기도 했는데, 그런 코드를 JUnit 안으로 옮기니 반복해서 실행하고 결과를 확인하기에 훨씬 편해졌습니다. 그 이후에는 Spring framework 의 통합테스트 기능을 이용한 테스트를 작성하기 시작했습니다. 특히 테스트 코드에서 DB 에 입력한 데이터를 자동으로 롤백시키는 기능이 DB 와 연동된 테스트를 할 때 유용했었습니다. 이후에 더 정교한 테스트를 하려다 보니 테스트용 객체를 만드는 프레임워크인 Mockito 를 사용하게 되었습니다. 테스트 코드를 먼저 작성하는 기법도 사용할 수 있게 되었습니다. 저는 실무에서 테스트코드를 작성하기 시작한 후부터 6 개월정도가 지나서야 Mockito 와 같은 라이브러리의 필요성을 느끼게 되었었습니다. 처음으로 시도한 프레임워크는 EasyMock 이었는데, 그 당시에는 프레임워크 제목처럼 전혀 Easy 하게 느껴지지 않았습니다. 근래에는 프레임워크의 발달로 이전보다는 테스트 코드를 작성하기도 쉬운 구조로 모듈을 구성하기도 쉽고 테스트 코드 작성 자체도 훨씬 편해졌습니다. 따라서 이제는 대부분 제가 겪었던 단계적인 과정을 훨씬 더 빠르게 경험하실 수 있으실 것이라 예상합니다.

자료구조/알고리즘

실무에서는 기본적인 알고리즘을 직접 구현하지는 않습니다. 이미 기본적인 SDK 나 많이 사용되는 라이브러리 솔루션에서 자주 쓰이는 자료구조, 알고리즘은 구현이 되어 있기 때문입니다. 하지만 그러한 이미 구현된 솔루션을 잘 선택하고 활용하기 위해서는 그 바탕이 되는 지식이 중요합니다.

JDK 의 Collection framework 의 소스를 볼 때에도 기본적인 자료구조에 대한 이해가 필요합니다. Java HashMap 은 어떻게 동작하는가?와 같은 깊이 있는 분석도 그런 배경지식을 바탕으로 하고 있습니다. 대용량 데이터를 어떻게 저장하고 탐색할지를 결정할 때도 자료구조는 중요합니다. LINE 소셜 네트워크 서비스의 아키텍처, SSD 는 소프트웨어 아키텍처를 어떻게 바꾸고 있는가? 의 기사에서 B-Tree, B+Tree 를 어떻게 활용했는지도 참고할 수 있습니다. 이미지 처리처럼, 특화된 분야의 알고리즘이 실무에서 응용되는 경우도 있습니다. PHOLAR 의 흔들림 보정 원리가 그 예입니다.

개발 프레임워크

JVM 생태계에서는 계속 Spring/Netty 기반의 프레임워크들이 꾸준한 발전을 하고 있다고 느껴집니다. 여러 개발팀에서는 고부하 처리를 위한 서버에는 Netty 를 바탕으로 UI 개발과 직접 호출되는 API 서버나 HTML 까지 그리는 서버 개발에는 Spring 계열이 많이 쓰이고 있습니다.
비동기 I/O 를 활용하는 서버 개발이 최근 몇 년 동안 서버개발자들 사이에서는 많은 관심을 끌고 있습니다. Java 생태계에서는 보통의 웹어플리케이션을 개발하는데까지는 비동기방식의 개발이 확산되지는 못했습니다. 여러가지 이유가 있겠는데, 비동기 개발 자체가 가지는 진입장벽과 기술적인 난이도가 있기도 합니다. RDB 를 호출하는 어플리케이션이 대부분인데 이를 뒷받침해야 할 비동기 JDBC 스펙의 구현체가 공식적으로 나오지 않은 탓도 있습니다. 최근 나온 Spring 5 에서 비동기 IO 개발을 지원하는 Webflux 모듈도 주목을 받고 있습니다.
모든 서버 어플리케이션이 비동기로 개발되어야 할 필요는 없다고 생각합니다. 그러나 트래픽이 많아서 동기/쓰레드 기반으로는 자원을 효율적으로 쓸 수 없는 시스템에서는 택할만한 선택지입니다. 네이버에서 고부하를 처리하는 플랫폼의 서버는 Netty/Vert.x 위에서 비동기 방식으로 개발된 것이 많습니다. 대용량 세션을 위한 로드밸런서 도 그 예입니다.

Serverless

Serverless 라는 용어가 자극적이기에 널리 퍼졌다는 생각도 듭니다. 주로 BaaS 와 FaaS 에 의존하는 아키텍처를 Severless 라고 주로 부릅니다. AWS 의 람다나 네이버 클래우드 플랫폼의 Cloud Functions 에 Servleless 아키텍처를 지원하는 플랫폼입니다. 요즘은 그 의미를 더 넓게도 쓰고 있습니다. 폭넓게는 어플리케이션 코드를 작성하는 개발자가 서버관리를 신경쓰지 않는 구조나 역할분담을 해당 어플리케이션 입장에서는 'Serverless'라고 부르기도 합니다. 외부로 공유하는 글에 Serverless 라는 용어를 쓰게 된다면 마틴파울러의 블로그에 올라온 https://martinfowler.com/bliki/Serverless.html 와 같은 글을 읽어볼 만합니다. Serverless 와 같이 인프라 환경이 고도로 자동화/추상화된 환경에서는 이제 전통적인 어플리케이션에서 했던 JVM, 커널 파라미터, 웹 서버 튜닝이 이제 필수 지식이 아니라고 생각할 수도 있습니다. 그러나 인프라의 기반 구조를 잘 이해하면 추상화된 서비스도 잘 쓸 수 있게 될 여지도 있습니다. 네이버와 같은 회사에서는 내부적으로 BaaS, FaaS 를 만들기도 하기 때문에 그런 프로젝트에 참여하는 개발자들에게는 최종 사용자 격인 개발자들보다 숨겨진 레이어에 대한 지식이 더 필요하기도 합니다.

클라이언트와 백엔드의 역할분담

이전에는 JSP 와 같이 HTML 를 서버사이드에서 생성하는 기술이 주로 쓰였습니다. 화면 전체를 다시 그릴 필요가 없는 요청은 Ajax 로 처리했지만, 서버 렌더링에 보조적으로 쓰이는 경향이 강했습니다. 하지만 요즘은 React.js 나 Vue.js 와 같은 프레임워크가 널리 쓰이면서 클라이언트 사이드에서 HTML 을 생성하는 비중이 높아지고 있습니다. https://d2.naver.com 도 React 를 이용해서 전적으로 클라이언트 사이드인 웹브라우저에서 HTML 을 그리고 있습니다. 선택지가 더 생김으로 인해서 어플리케이션이나 페이지의 특성에 따라서 어느 방식이 적합할지 이전보다 더 많이 고민하게 되었습니다. 사용자가 페이지에 처음에 진입할 때는 서버에서, 그 이후로는 클라이언트에서 HTML 을 그리는 Universal Rendering 도 최근 많이 시도되고 있습니다.
HTML 렌더링을 어디에서 할지는 개발자 간의 역할분담에도 영향을 미칩니다. 100% 클라이언트 사이드 렌더링을 하는 구조는 서버 개발자가 HTTP API 개발에만 전념할 수 있다는 장점이 있습니다. 반면 전문

Front End 개발자의 작업이 더 많아지는 부담도 생깁니다. 제가 속한 조직에서는 백엔드 개발자들도 React.js 나 Vue.js 를 이용한 UI 개발자 참여함으로써 그런 단점을 보완하고 있습니다.

클라이언트를 위한 API 설계

앞서 언급한 REST 스타일의 일부를 차용하더라도 HTTP API 의 설계의 많은 부분은 매번 프로젝트마다 고민해야할 점이 많습니다. 예를 들면 단순 CRUD(입력/조회/수정/삭제) API 는 각각 POST/GET/PUT/DELELE 의 HTTP 메서드로 연결시키더라도 이를 벗어난 기능들은 어떻게 설계해야할지 명확하지 않은 경우가 많습니다. 페이징 처리를 할 때나 복잡한 검색조건이 있을 때의 파라미터 표현방식에서도 서비스마다 다르게 정의하고 있습니다. 그리고 API 에서 선택적으로 반환할 속성과 아닐 속성을 지정하는 방식도 다양합니다. 아래의 스펙들이 보다 구체적인 API 설계안들을 제안하기는 합니다. 그러나 아직 어떤 상세한 스펙이 지배적으로 많이 쓰이고 있지는 않은 상황입니다.

Microsoft REST API Guidelines

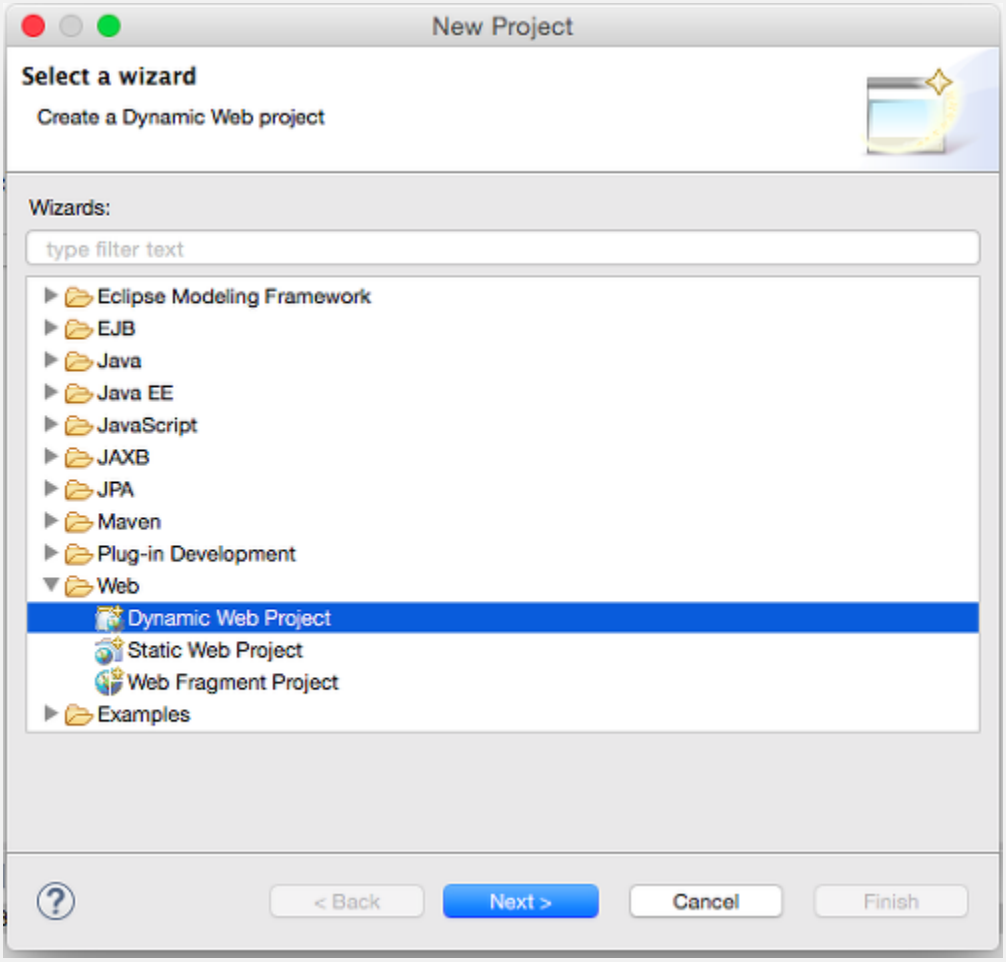
JSON API 스펙

클라이언트 사이드에서 UI 렌더링을 하는 경우가 늘어나면서도 HTTP API 를 더 다양하고 정교하게 사용하고자 하는 필요성이 늘었습니다. 이러한 흐름 속에서 Facebook 에서는 GraphQL 을 내 놓았고, Github 에서도 이를 도입하는 등 점점 확산되고 있는 추세입니다. 현시점에서 기존의 소위 REST 스타일과 유사한 틀에서 나름대로의 관례를 정립해서 최선을 추구할지, GraphQL 을 도입할지도 고민되는 선택 중의 하나입니다.

Chapter 2. 웹 애플리케이션 개발 환경 구축

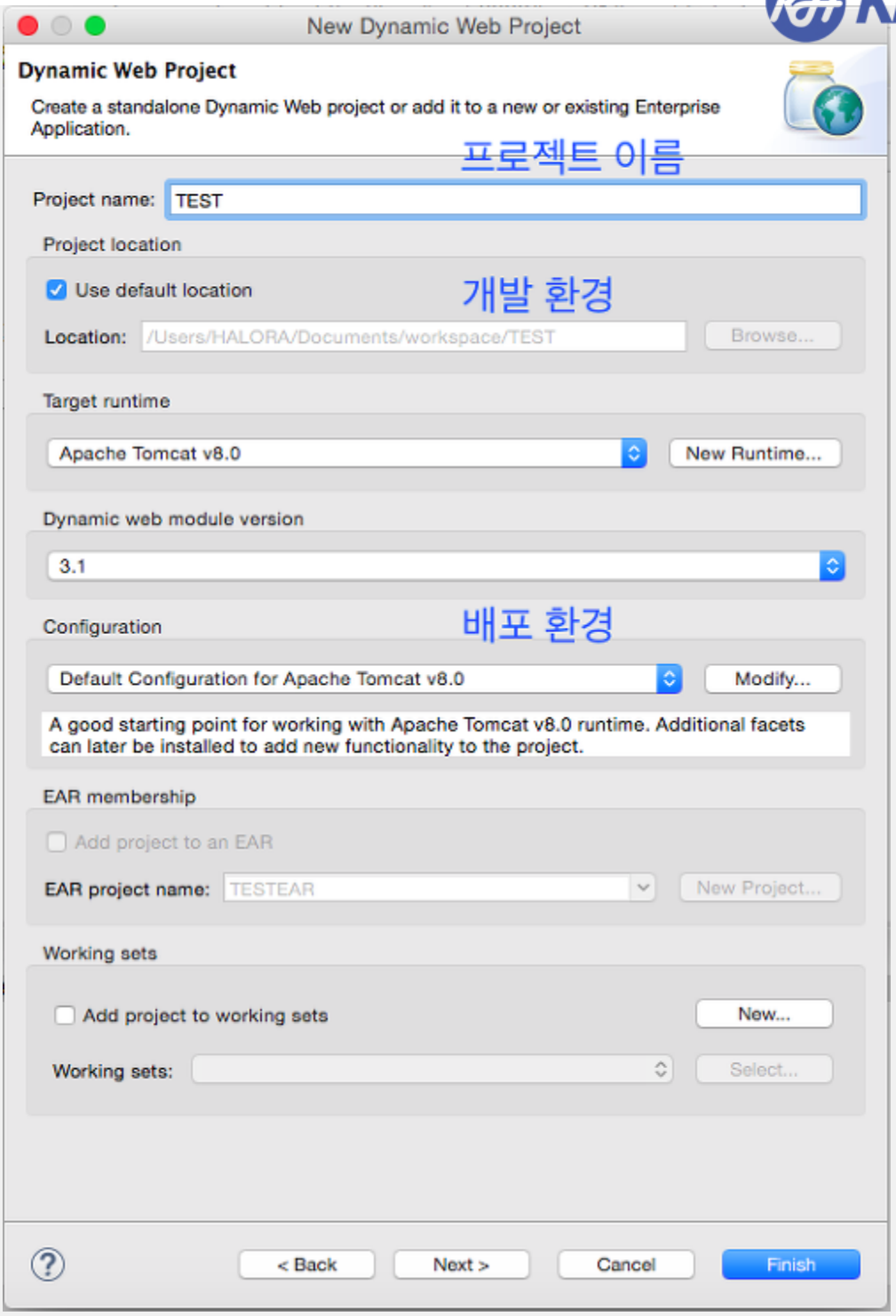
프로젝트 생성
Dynamic Web Project

1. Project Type 지정: Dynamic Web Project



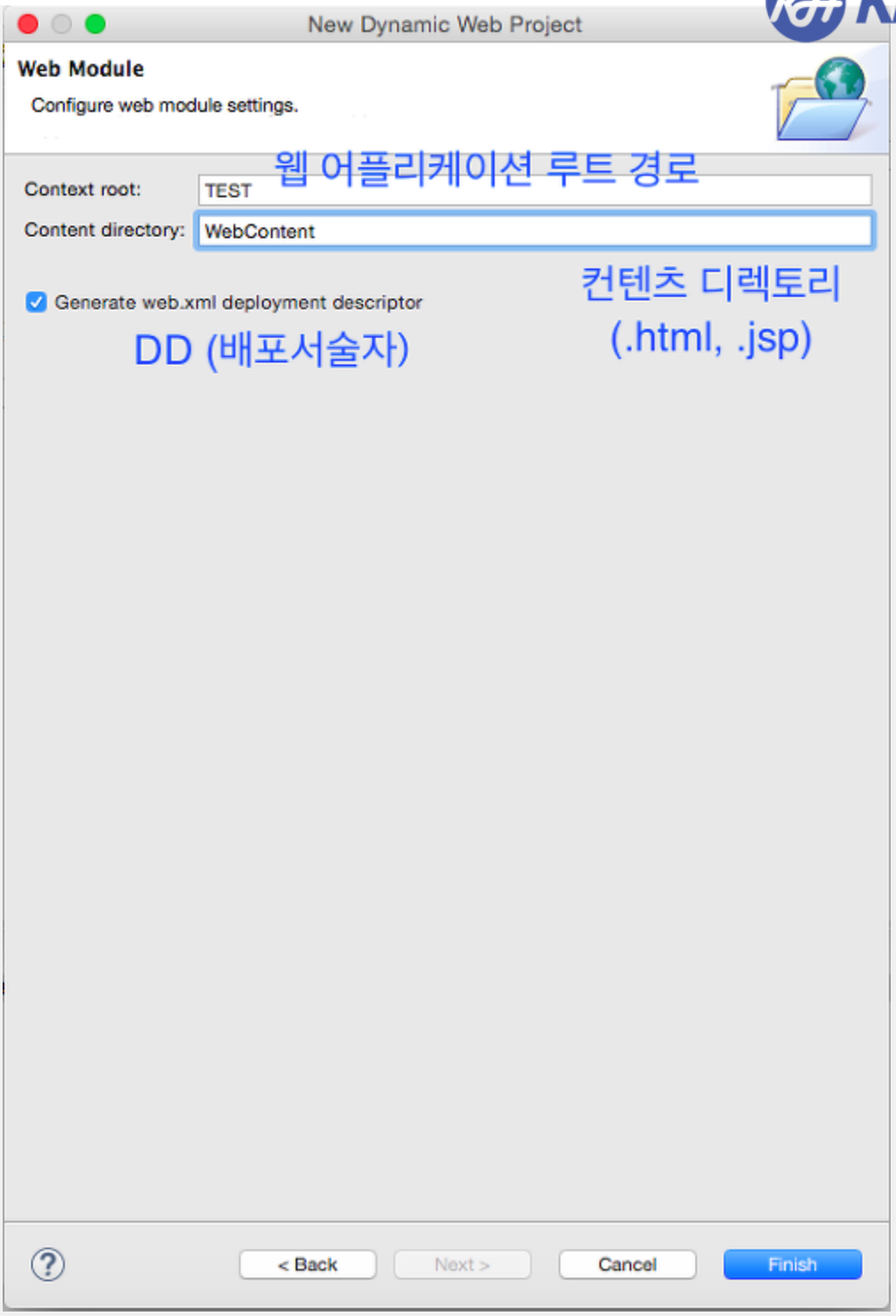
2. 프로젝트 생성

- 프로젝트 이름
- 프로젝트 개발환경
- 프로젝트 배포 서버 지정



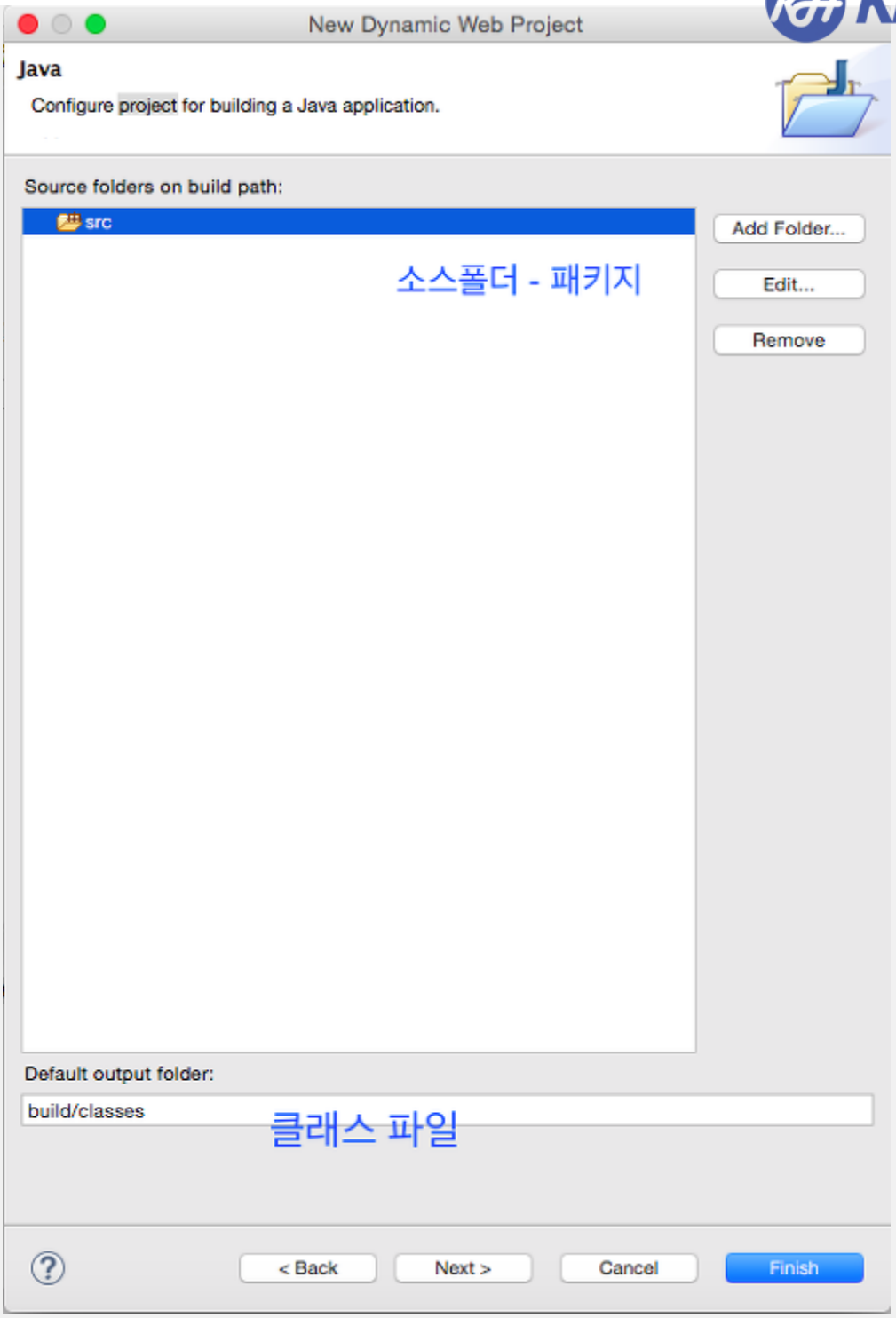
3. Web Module 세팅

- 웹모듈이란? 배포가능한, 사용가능한 웹리소스의 최소단위
- Context Root: 웹 어플리케이션의 Document Root Directory
- Content Directory: Static Web Content File 이 위치할 경로 (.html, .jsp)
- DD: 배포서술자 Deployment Descriptor



4. 패키지 생성

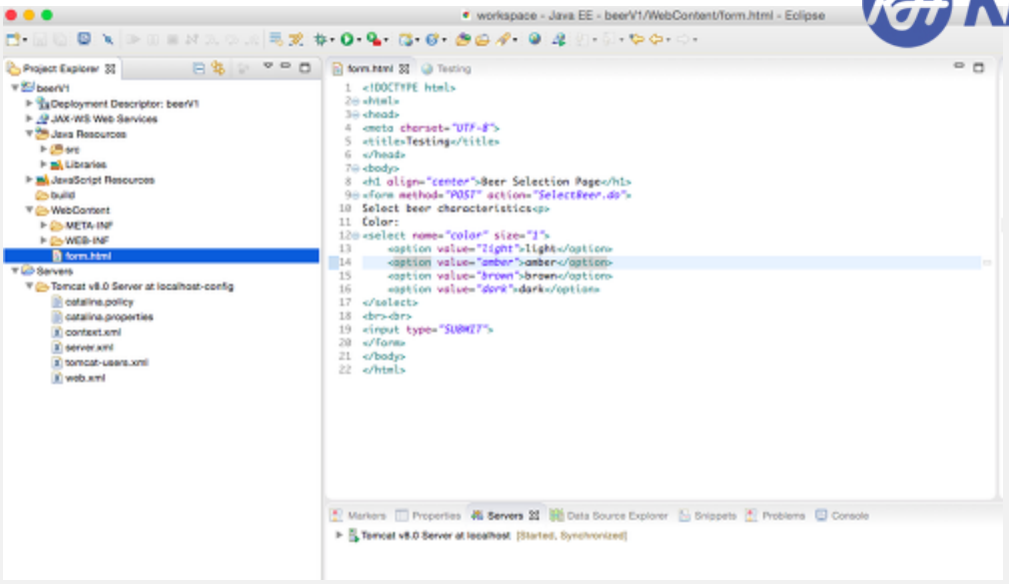
- 개발환경: server-side 클래스들을 개발할 소스폴더 (패키지) 를 생성합니다. servlet, 유틸리티 클래스, JavaBeans 컴포넌트 등이 위치합니다.
- 배포환경: 소스폴더에 대응되는 컴파일 완료된 클래스 파일 경로를 지정합니다.



로컬 실행 (+테스트)

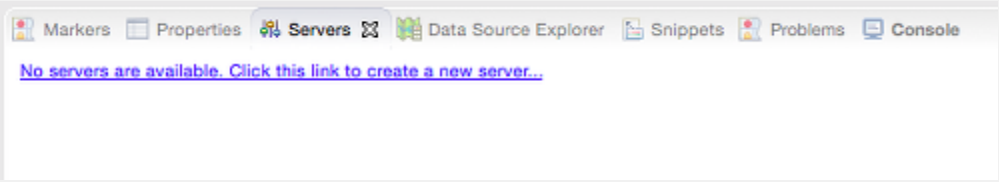
1. .html 웹콘텐츠 실행

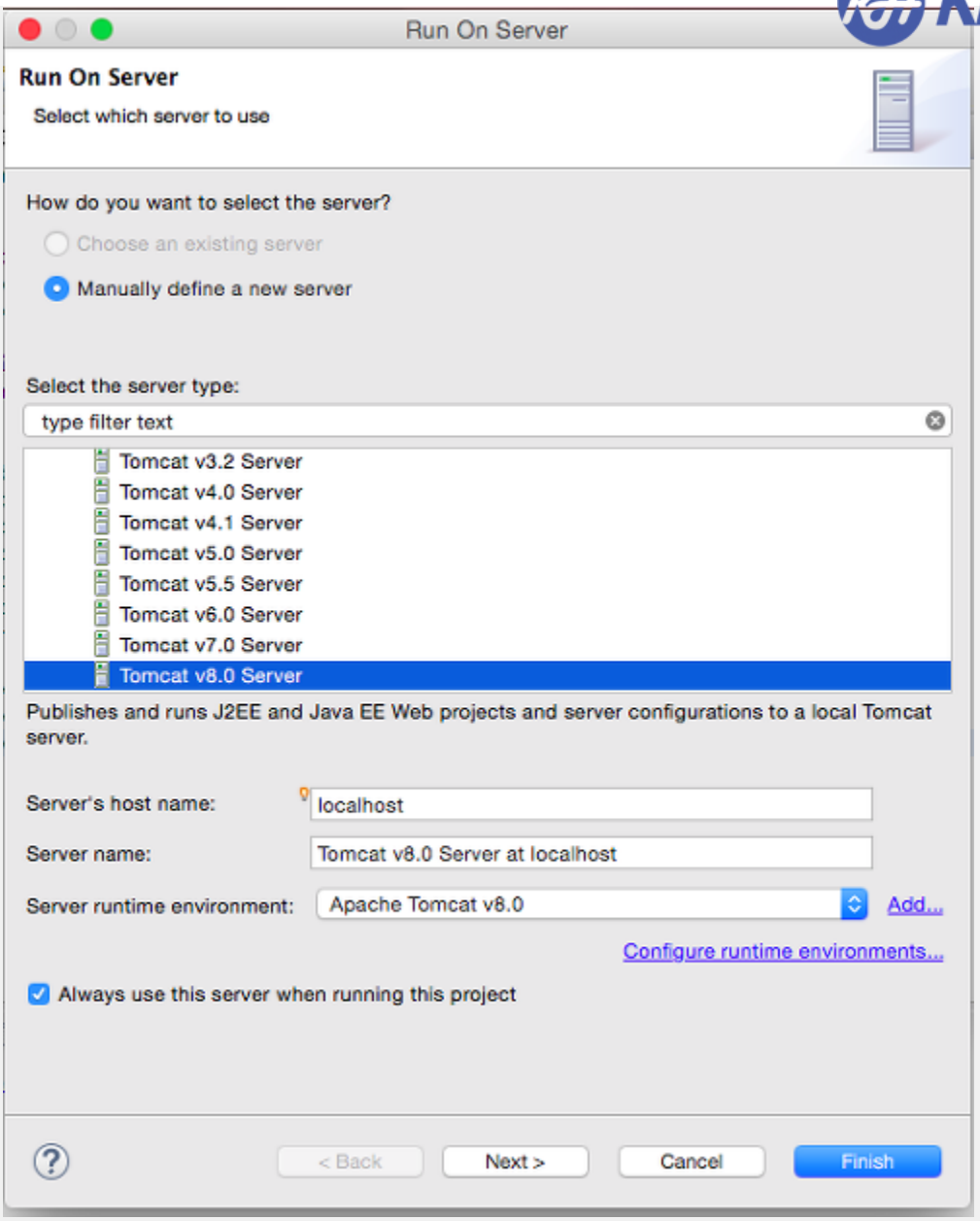
- WebContent Directory (ContentDirectory) 에 .html 생성



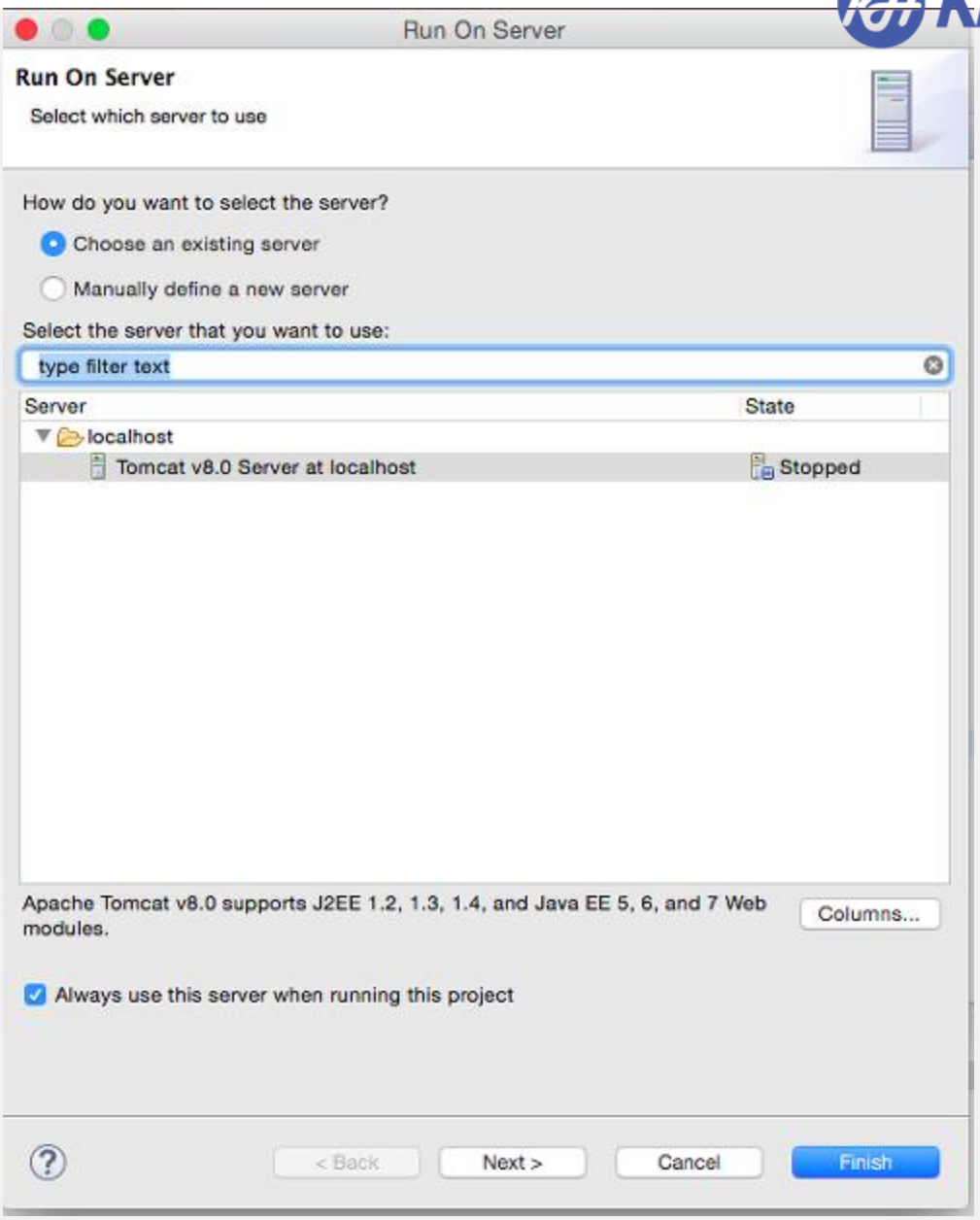
2. 서버 세팅

- 구동 서버 선택

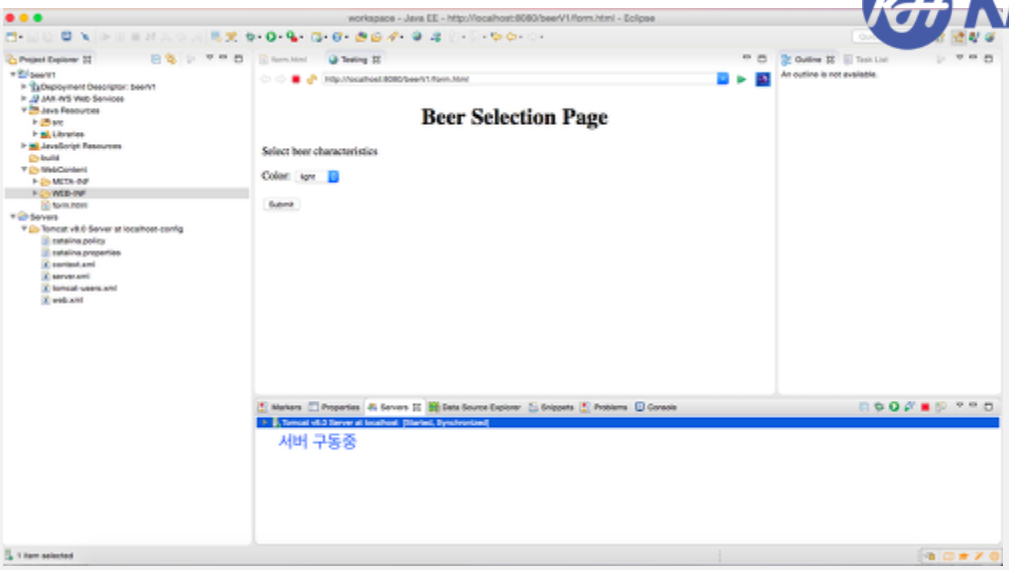




3. 구동할 애플리케이션 선택

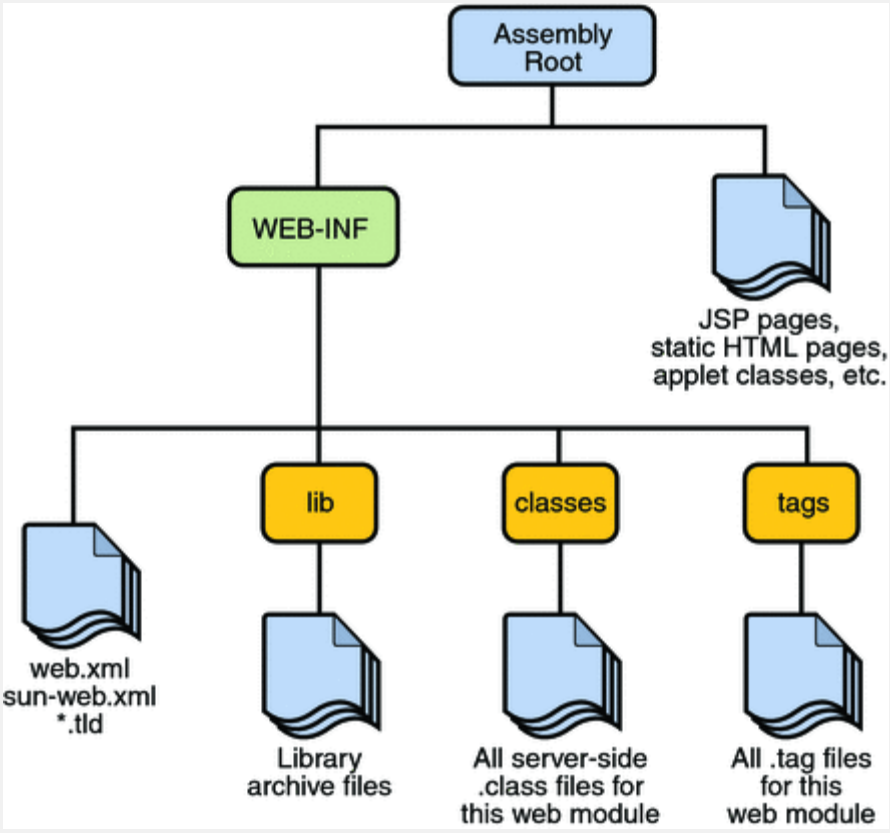


4. Run : 웹페이지 호출완료



정리하자면-

프로젝트에 웹 리소스를 정적인 파일 / 동적인 파일을 구분하여 위치시킵니다. WEB-INF 에 DD, lib, 클래스파일, 커스텀태그 등이 위치합니다. 이번 포스트에서는 정적인 파일까지 위치완료! 다음 포스트에서는- 간단한 동적 웹리소스 - 서블릿 클래스도 동작시켜보겠습니다



웹 모듈 구조 Web-Module-Structure

Chapter 3. 형상관리도구 git 사용

1 Git이란

트렁크, 브랜치 개념을 지닌 SVN은 중앙 집중형 형상 관리 기법을 사용하는 소프트웨어인 반면, Git은 로컬 저장소(Local Repository)와 원격 저장소(Remote Repository)로 나뉘어 별도 브랜치를 생성 후 merge하는 분산 소스형 형상 관리 기법을 통해 버전 관리를 수행하는 소프트웨어이다.

또한 원본 소스를 저장할 공유 Server가 필요한 SVN과는 다르게 Github, Bitbucket이라는 소스를 저장할 수 있는 원격 저장소를 제공하는 웹 사이트가 있어, 팀 프로젝트 수행 시 비용이 적게 드는 장점이 있다.

Git 에서는 원격저장소의 master를 최대한 건드리지 않고, **각자 브랜치를 생성하여 작업** 후 원본(master / origin)에 merge 하는 방식을 쓰기 때문에, SVN 방식과 헷갈리지 않도록 하자.

2 Git 설치하기

- Git 직접 설치

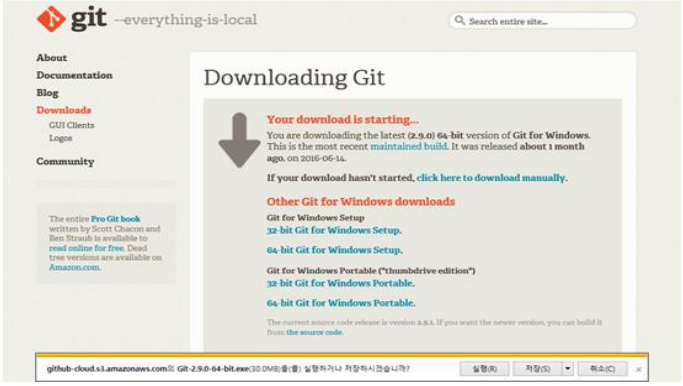
공식 사이트인 https://git-scm.com/ 에서 오른쪽 화면에 보이는 'Downloads for Windows' 을 클릭하여 다운로드 페이지로 이동한다.



※ 현재 접속중인 컴퓨터의 운영체제에 따라 자동으로 <Downloads for Linux>, <Downloads for Mac> 등으로 달라진다.

※ Git 에 대해 더 알고 싶다면 https://git-scm.com/book/ko/v2 사이트를 참조하면 된다.

아래쪽에 자동으로 자신의 버전과 맞는 최신 버전이 뜬다. 다른 버전을 다운받고 싶으면 취소를 누르고 원하는 버전을 직접 다운로드 받으면 된다.



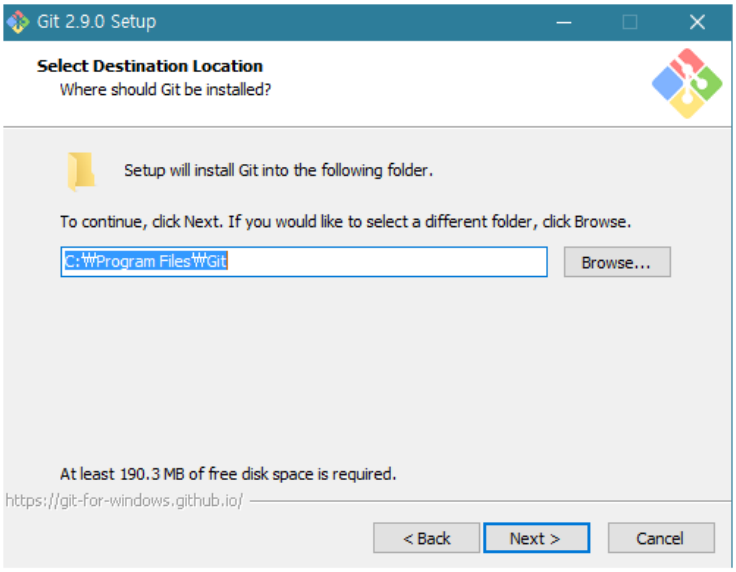
※ Git은 안정성이 검증되지 않았기 때문에 실제 다운로드 페이지에서 표기된 최신 버전보다 더 옛날 버전을 사용한다.

1. 윈도우에서 설치

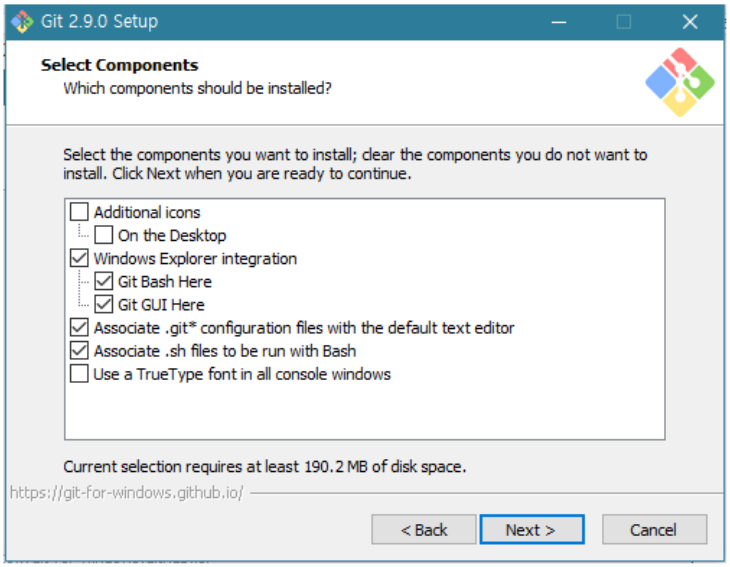
- <Next> 클릭



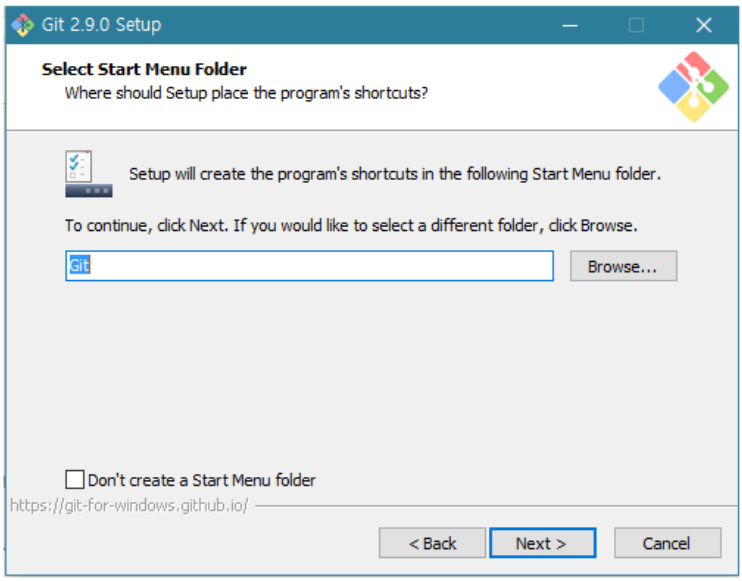
- <Next> 클릭



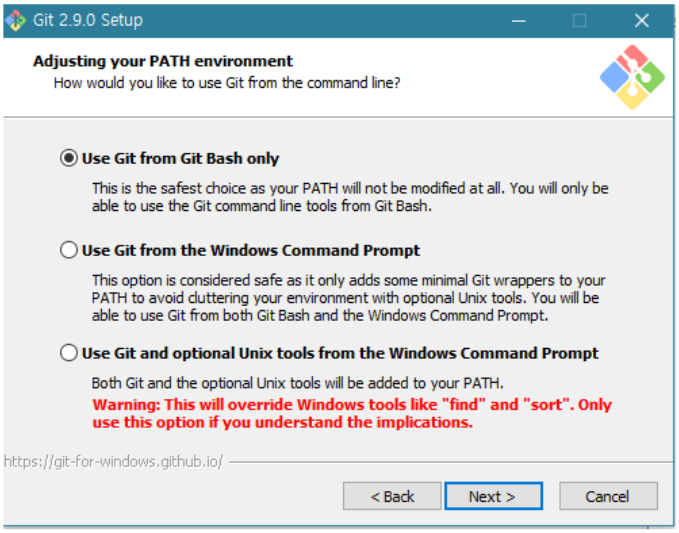
• 추가로 추가할 옵션이 있다면 선택하고 Next 클릭



• <Next> 클릭

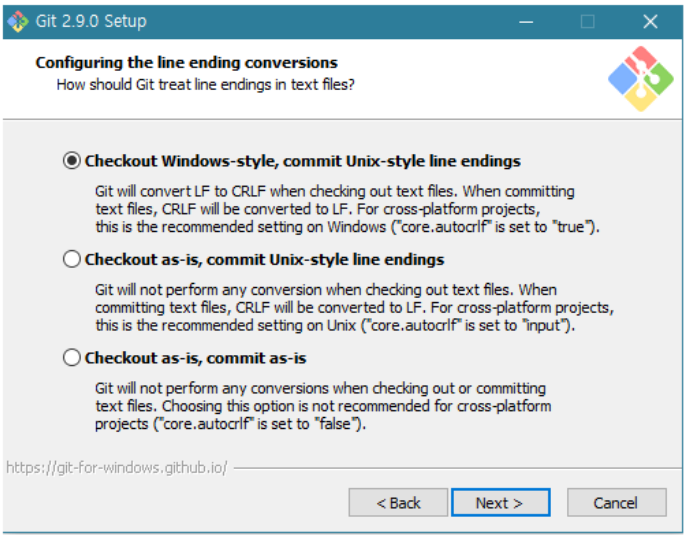


• 환경 변수를 맨 위로 체크하고 <Next> 클릭

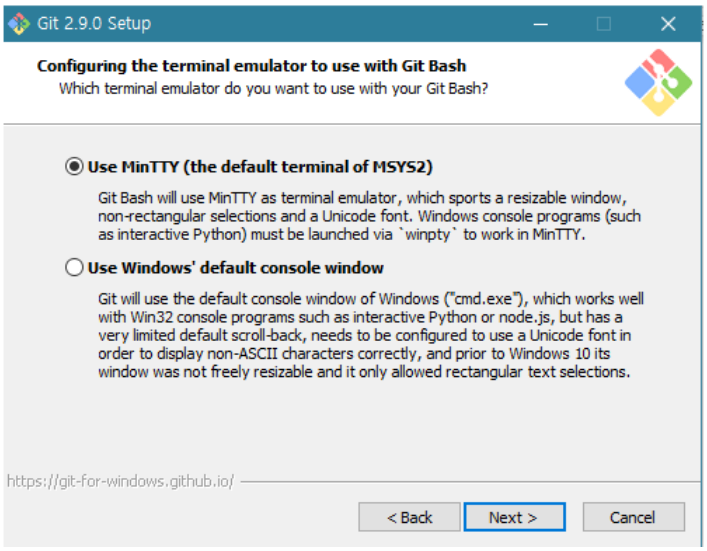


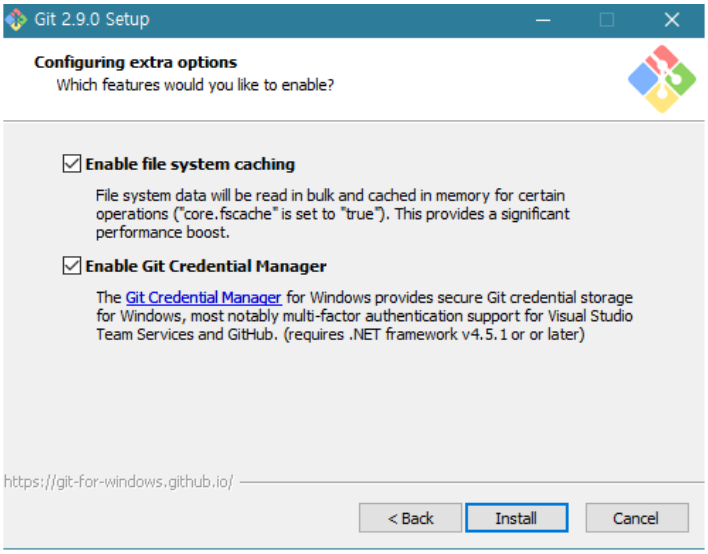
※ Use Git from Git Bash only : 환경 변수를 변경하지 않고 Git bash 커맨드 라인 도구에서만 실행한다.
※ Use Git from Windows Command Prompt : 윈도우 커맨드 프롬프트에서 Git을 실행할 수 있는 최소한의 내용을 환경 변수에 추가해 설치한다.
※ Use Git from optional Unix tools from the Windows Command Prompt : Git과 부수적인 UNIX 도구들을 모두 윈도우 환경 변수에 추가한다.

• Git에서 커밋할 때 작성하는 라인 끝을 어떻게 처리할 지 결정한다. 기본으로 선택된 첫 번째 옵션으로 두고 <Next> 클릭

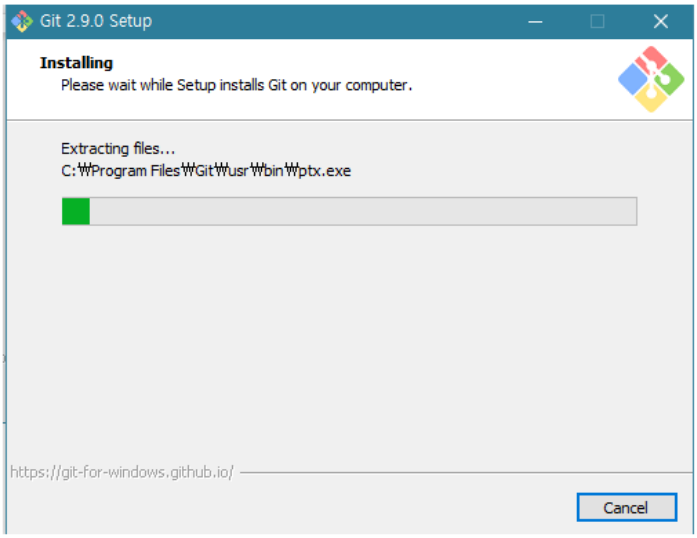


• <Next> 클릭


Git 2.9.0 Setup
Configuring the terminal emulator to use with Git Bash
Which terminal emulator do you want to use with your Git Bash?
Use MinTTY (the default terminal of MSYS2)
Git Bash will use MinTTY as terminal emulator, which sports a resizable window, non-rectangular selections and a Unicode font. Windows console programs (such as interactive Python) must be launched via `winpty` to work in MinTTY.
Use Windows' default console window
Git will use the default console window of Windows ("cmd.exe"), which works well with Win32 console programs such as interactive Python or node.js, but has a very limited default scroll-back, needs to be configured to use a Unicode font in order to display non-ASCII characters correctly, and prior to Windows 10 its window was not freely resizable and it only allowed rectangular text selections.
https://git-for-windows.github.io/
< Back
Next >
Cancel

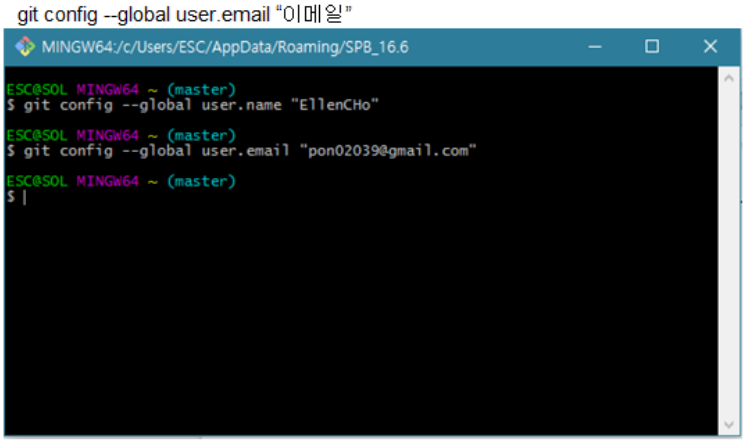

Git 2.9.0 Setup
Configuring extra options
Which features would you like to enable?
Enable file system caching
File system data will be read in bulk and cached in memory for certain operations ("core.fscache" is set to "true"). This provides a significant performance boost.
Enable Git Credential Manager
The Git Credential Manager for Windows provides secure Git credential storage for Windows, most notably multi-factor authentication support for Visual Studio Team Services and GitHub. (requires .NET framework v4.5.1 or or later)
https://git-for-windows.github.io/
< Back
Install
Cancel

• 설치


Git 2.9.0 Setup
Installing
Please wait while Setup installs Git on your computer.
Extracting files...
C:₩Program Files₩Git₩usr₩bin₩ptx.exe
https://git-for-windows.github.io/
Cancel

- 시작메뉴 -> Git -> Git Bash 를 실행하여 사용자 이름과 이메일을 설정한다.

git config --global user.name "사용자 이름"
git config --global user.email "이메일"



- 설정 완료

- Github 사용하여 Git 저장소 만들기

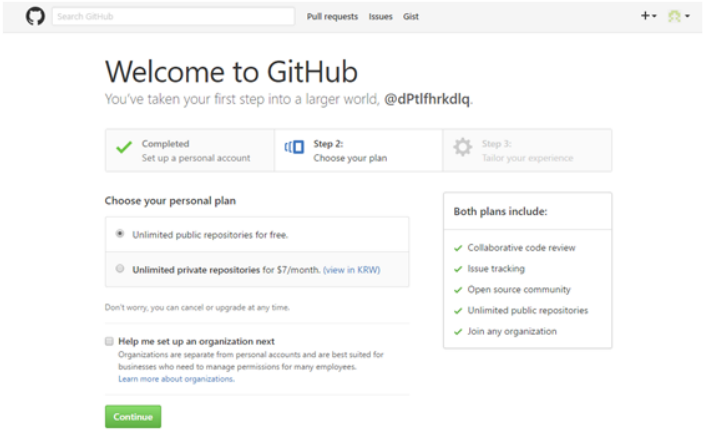
GitHub는 원격 저장소를 제공하며 여러가지 프로젝트 진행을 원활하게 하는 도구를 함께 제공한다.

1. 회원 가입

- 공식 사이트인 https://github.com/ 에 접속해서 오른쪽에 사용자 아이디, 이메일, 비밀번호를 치고 <Sign up for GitHub>를 클릭한다.



- 지불하는 비용에 따라 비공개 저장소를 사용할 수 있다. 무료로 GibHub를 이용한다면 모든 저장소는 공개 저장소가 된다. 맨 아래 기본으로 선택된 'Free'항목으로 그대로 두고 <Finish sign up> 버튼을 누른다.



※ 'Help me set up an organization next' 체크 박스는 많은 사람이 협업할 때 팀을 만들어서 활동하도록 설정하겠다는 의미이다.

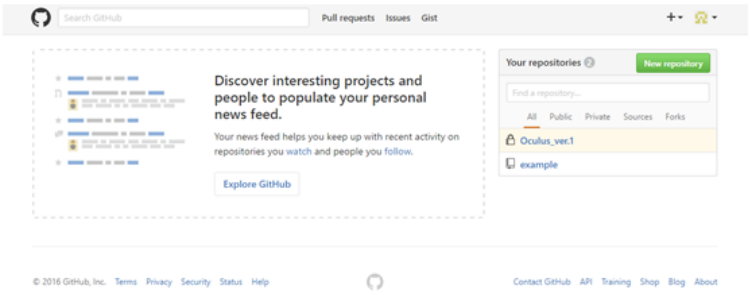
- 회원 가입할 때 입력했던 이메일을 확인해서 GibHub 가입을 인증한다.

- 회원 가입 완료

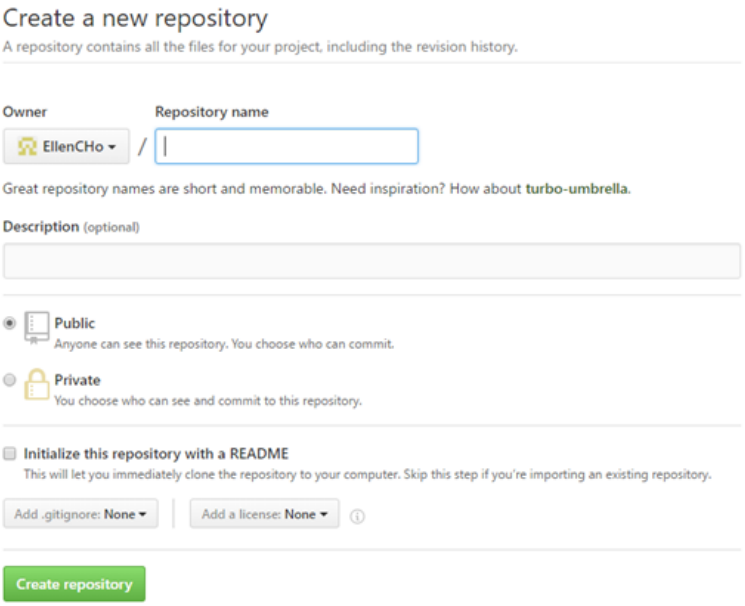
2. 원격 저장소 생성

- 포크 : 다른 사람의 저장소를 내 저장소로 복사하는 기능
- 풀 리퀘스트 : 포크한 저장소를 수정해 다시 원본 저장소에 병합해달라는 요청을 보내 사용자 사이의 상호작용을 일으키는 기능
- 이슈 : 저장소 안에서 사용자들 사이의 문제를 논의하는 기능
- 위키 : 저장소와 관련된 체계적인 기록을 남기는 기능

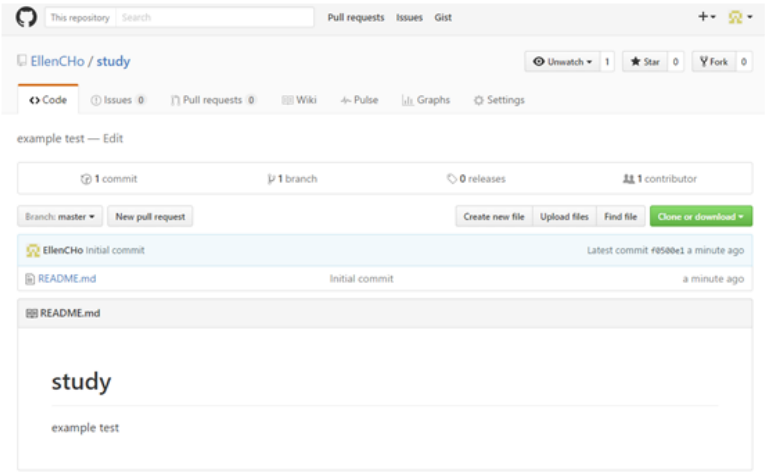
- 오른쪽 아래에 있는 <New repository>를 클릭해서 새 원격 저장소를 생성한다.



- 아래 항목을 참고해서 빈칸을 채운다.
  - Owner : 사용자의 아이가 표시된다. 협업 환경에서는 다른 사용자의 아이디를 지정 할 수 있다.
  - Repository name : 새로 생성할 원격 저장소의 이름을 입력한다. 가능하면 로컬 환경에서 작업할 Git 프로젝트 디렉터리 이름과 같게 하는 것이 좋다.
  - Description : 꼭 작성할 필요는 없는 항목이지만 생성한 원격 저장소가 어떤 역할을 하는 지를 간단하게 적어두면 원격 저장소가 많아졌을 때 구분하기 쉽다.
  - Public/Private : 원격 저장소의 공개 여부를 선택하는 옵션이다. 무료 사용자는 Public만 사용 가능하다.
  - Initialize this repository with a README : 기본적으로는 체크 표시를 해준다. 체크해주면 GitHub에서 생성한 원격 저장소를 바로 로컬 저장소에 복사해서 가져올 수 있다. 또한 '저장소 이름'과 'Description' 항목의 내용을 담은 README.md 파일을 생성한다.
  - Add .gitignore : 원격 저장소에 포함하지 않을 파일들의 목록을 만들 때 사용한다. 지금 당장은 사용할 필요가 없으니 'none' 상태로 둔다.
  - Add a license : 원격 저장소에 저장할 프로젝트가 어떤 라이선스에 속할지를 선택한다. 지금 당장은 사용할 필요가 없으니 'none' 상태로 둔다.



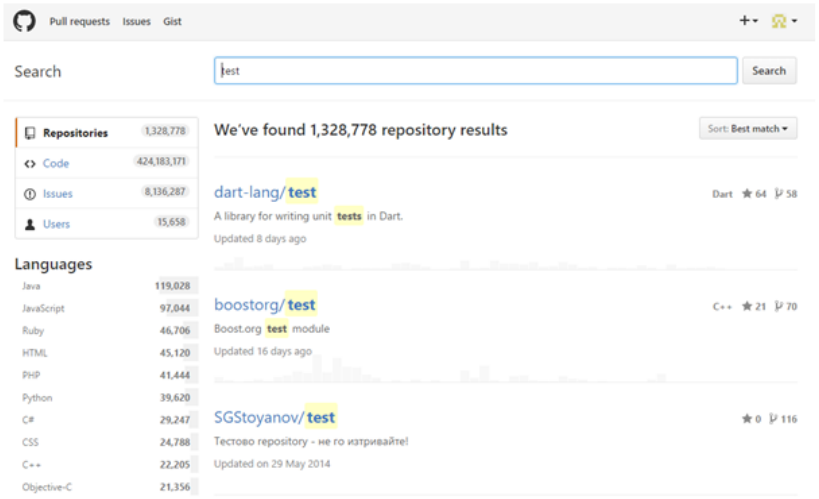
- <Create repository>를 클릭하면 아래와 같은 원격저장소가 생성된다.



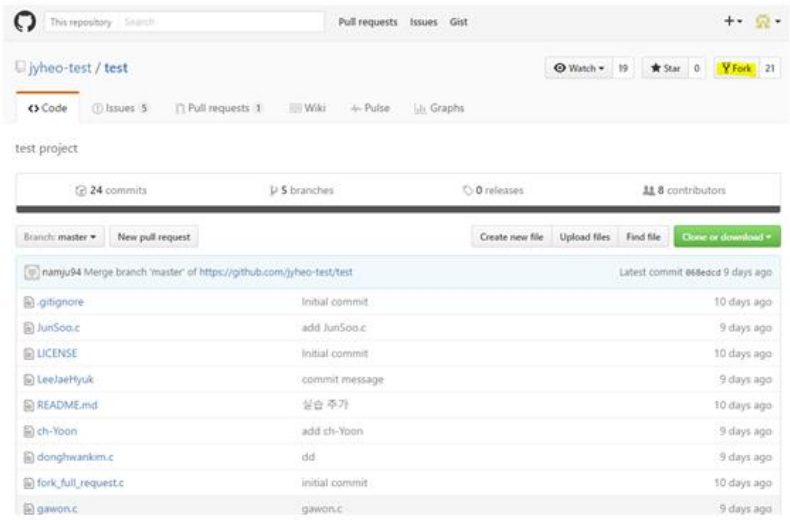
3. 포크

다른 사람의 원격 저장소를 내 계정으로 복사하는 방법을 포크라고 한다. 만약 포크하지 않는다면 내 것이 아닌 저장소 다시말하면 쓰기 권한이 없는 원격 저장소를 사용하는 것이므로 자유롭게 파일을 생성하거나 수정하여 원격 저장소에 반영하는 것이 불가능하다.

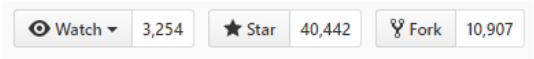
- 원격 저장소 검색(ex. test)



- 오른쪽에 있는 <Fork>를 클릭하면 내 원격 저장소에 저장됨



4. GitHub 원격 저장소의 구조

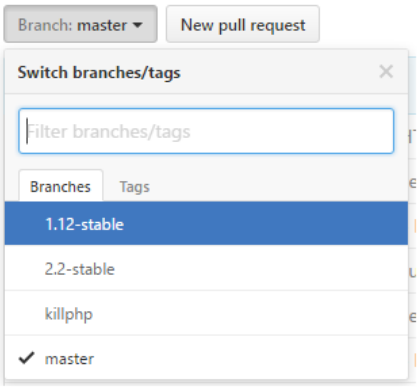


- Watch : 해당 버튼을 클릭하면 원격 저장소의 활동 내역을 사용자에게 알려준다. 댓글이나 이슈 등에서 언급될 때만 알려주는 Not Watching, 모든 활동 내역을 알려주는 Watching, 모든 알림을 무시하는 Ignoring을 선택할 수 있다. 오른쪽 숫자는 현재 활동 내역을 보고 있는 사람의 수이다.
- Star : 해당 원격 저장소에 관심이 있을 때 클릭하면 된다. 오른쪽 숫자는 관심이 있는 사람의 수를 나타낸다.
- Fork : 해당 버튼을 클릭하면 원격 저장소를 포크한다. 오른쪽은 포크한 사람의 수를 나타낸다.



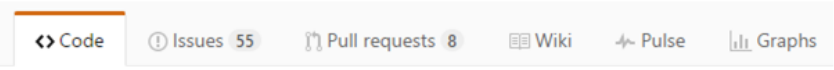
- Description : 원격 저장소를 설명하는 메시지가 나타난다.
- Commits : 원격 저장소의 총 커밋 수를 나타낸다.
- Branches : 원격 저장소의 브랜치 수를 나타낸다.
- releases : 원격 저장소의 태그 수를 나타낸다. 주로 특정 버전에 표식을 주고 싶을 때 사용한다. 이 표식을 통해서 특정 버전을 다운로드할 수 있다.
- Contributor : 원격 저장소에 커밋 혹은 풀 리퀘스트가 받아들여진 사용자 수이다. 이 저장소가 오픈 소스라면 이 오픈 소스에 공헌한 사람 수라고 생각해도 된다.

※ 커밋은 뒤에 나오지만 저장할 때마다 끼워넣는 책갈피라고 생각하면 쉽다. 지금 저장한 버전이 무엇을 수정했는 지, 무슨 내용을 담고 있는 지 설명해주는 지표이다.



- Current branch : 원하는 브랜치를 선택하는 기능

- Current branch : 원하는 브랜치를 선택하는 기능



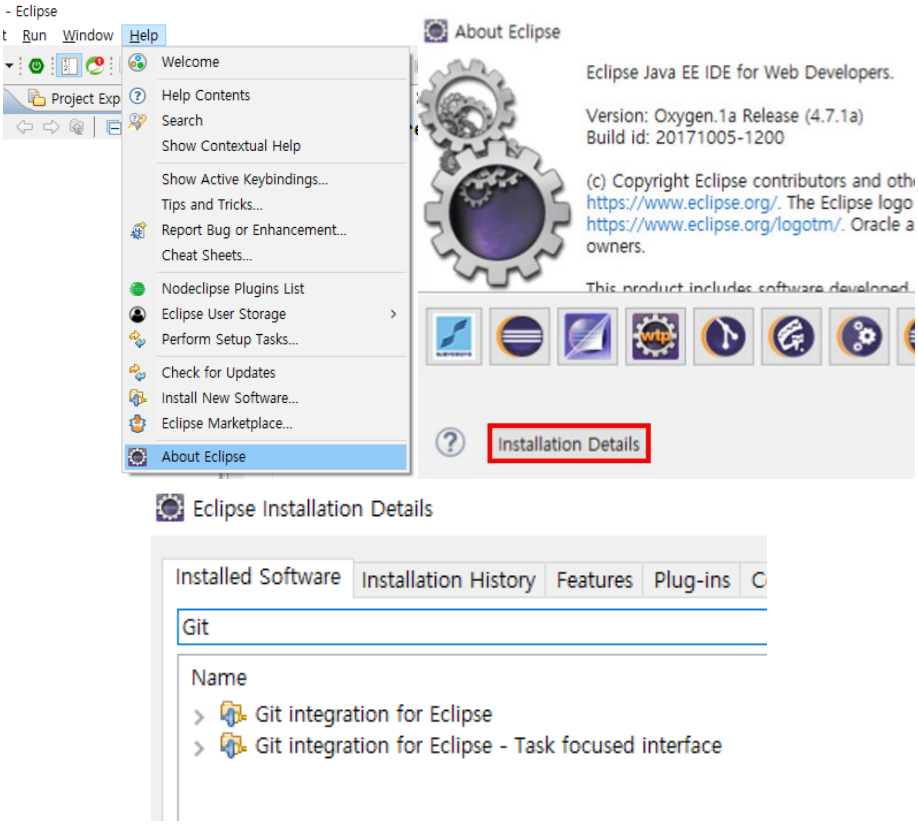
- Code : 해당 원격 저장소의 루트 디렉토리로 이동한다. 어떤 경로에 있더라도 루트 디렉터리로 이동한다.
- Issues : 해당 원격 저장소의 주요 이슈 사항을 기재한 후 관리합니다. 게시판 형태이며 댓글 형태로 토론이 이뤄지기도 한다. 보통은 문제점이나 개선점을 얘기한다.
- Pull Requests : 풀 리퀘스트 전체 목록을 모아서 보여준다. Issues와 마찬가지로 목록마다 댓글 형태로 토론할 수 있다. 보통 오른쪽에 있는 숫자는 현재 요청이 온 풀 리퀘스트를 받아들일 것인지에 대한 논의가 몇개인지 알려주는 기능이다.
- Wiki : 공유할 정보나 개발 문서, 참고 자료 등을 작성하기 위한 기능입니다. 마크 다운과 위키위키 문법을 사용한다.
- Pulse : 해당 원격 저장소의 최근 변경 내역을 확인할 수 있다. 최대 한 달까지의 변경 내역을 확인할 수 있으며 풀 리퀘스트 몇 개중에 몇 개가 받아들여졌나, 이슈는 몇 개가 있고 몇 개가 해결되었나, 해당 풀 리퀘스트와 이슈에 관련된 활동을 볼 수 있다.
- Graphs : 공헌자의 공헌 내역, 커밋 수 등 해당 저장소의 활동 내역을 그래프화해서 보여준다.

3 프로젝트에 Git 적용하기

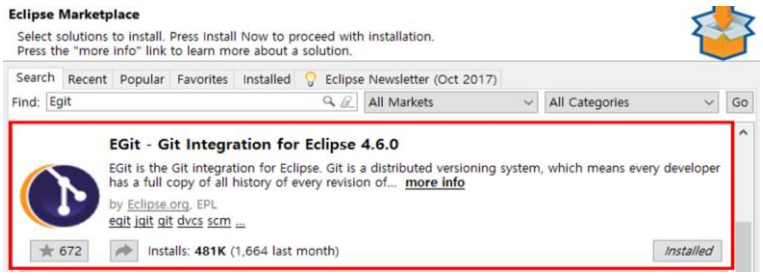
사용의 편의에 따라 본 문서에서는 Github를 사용하는 예시를 주로 다루도록 하겠다.

먼저 이클립스에서 git과 관련된 플러그인이 설치되어 있는지 확인하자.

이클립스 상단의 Help > About Eclipse > Installation Details 에서 Git으로 검색 후 하단의 항목이 뜨면 별도의 git 플러그인 설치는 필요 없다.

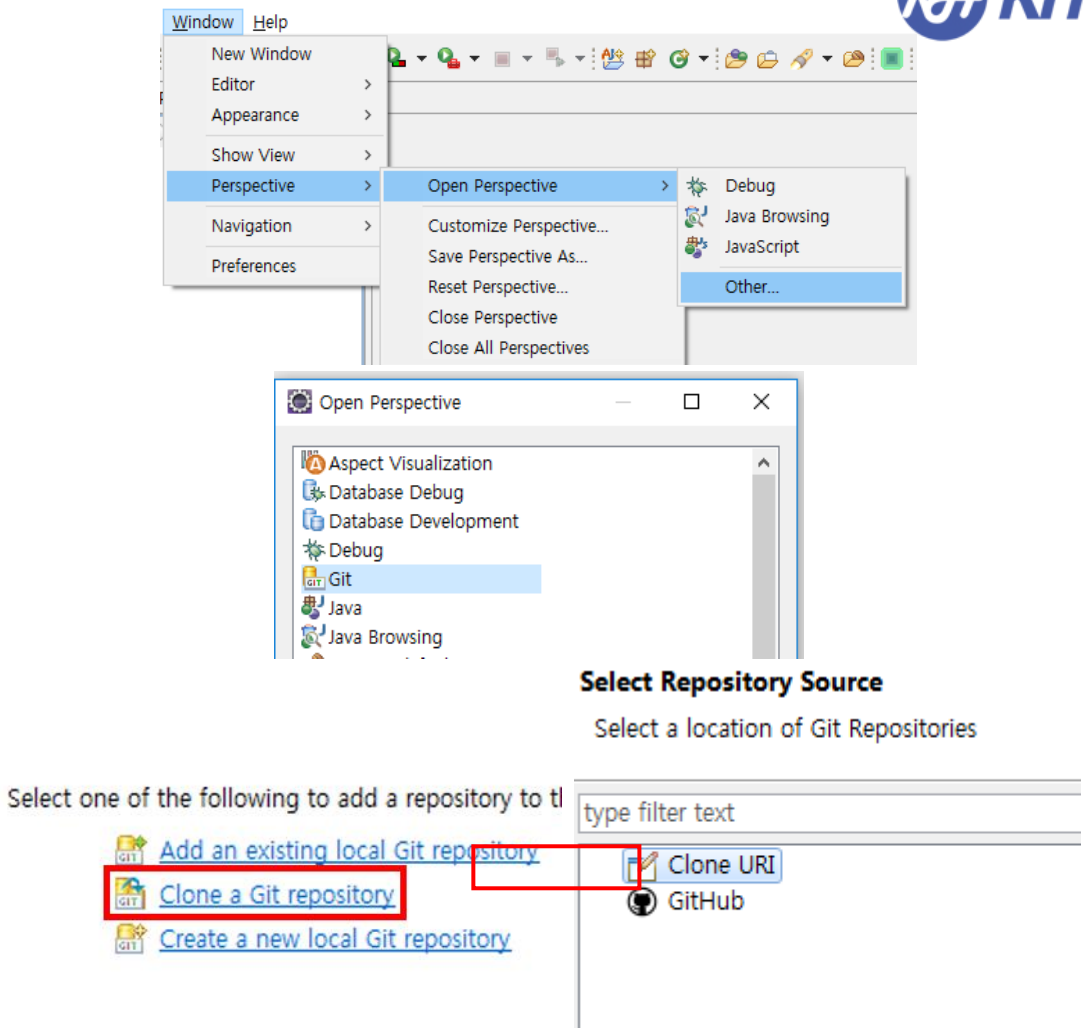


만약 상단의 Git 플러그인 설치가 안되어 있을 경우, Help > Eclipse Marketplace를 통해 EGit을 받도록 하자.

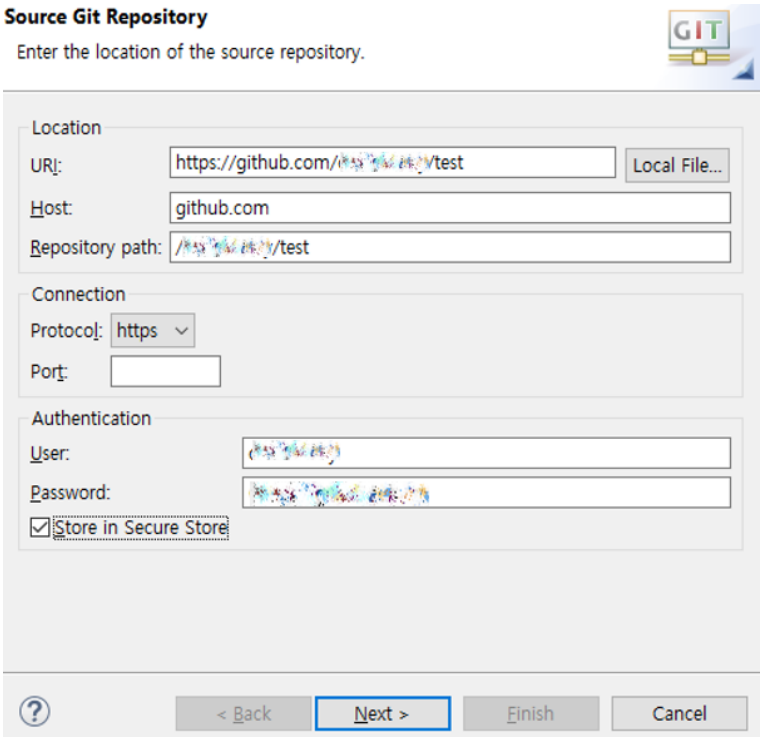


이제 Github를 통해 생성한 저장소를 연동하도록 하겠다. 여기서는 임시로 test라는 저장소를 생성했다고 가정한다. 만약 **ignore할 설정이 필요하다면, 저장소 생성 시 설정**해야 함을 참고하도록 하자.

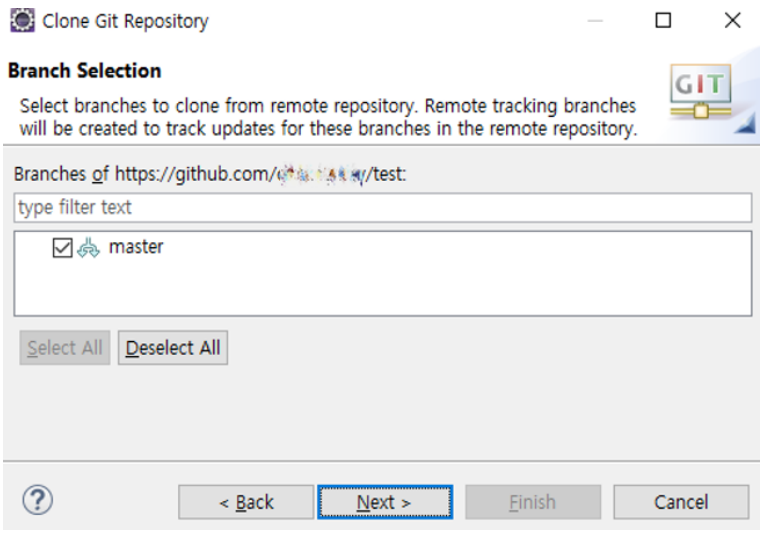
먼저 상단의 Windows > perspective > Open Perspective > Other 로 접속하여 Github와 연동 설정을 추가한다.

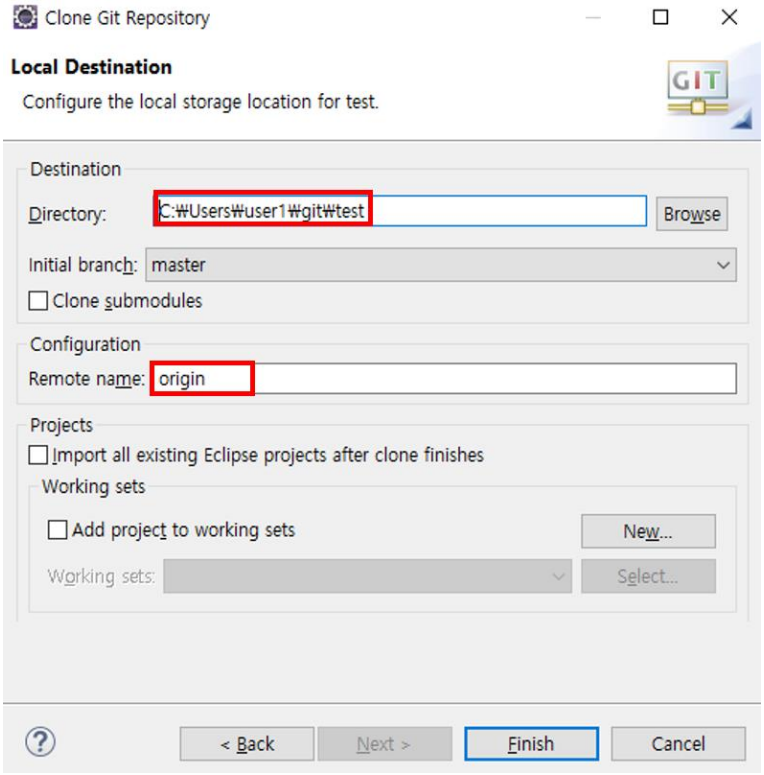


해당 창이 팝업되면 github에 만들어 둔 저장소의 주소를 복사해서 붙여 넣고, ID 와 Password 설정 후 Next를 누른다.

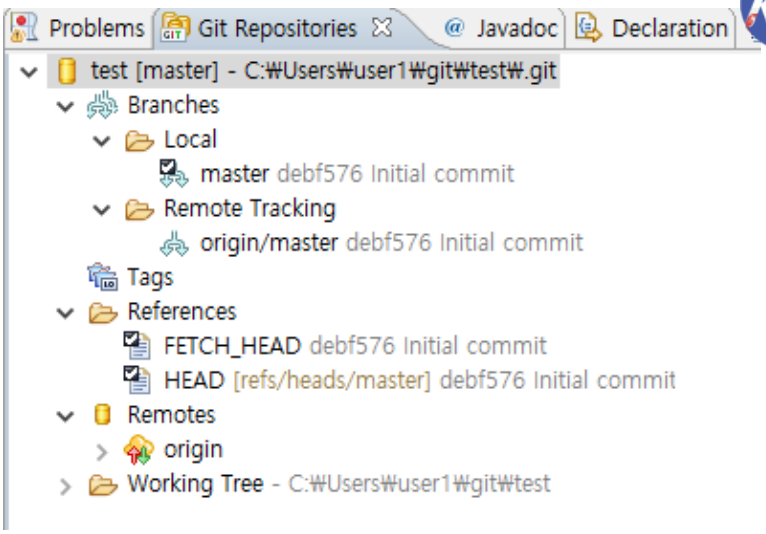


Master 브랜치를 확인하고 next를 클릭한다.

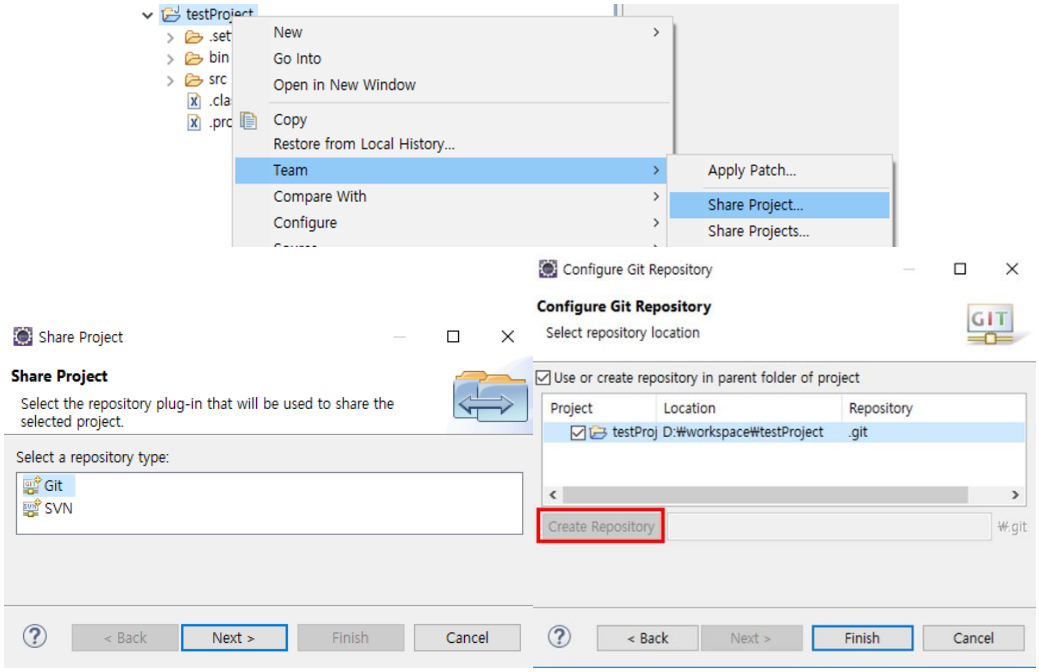




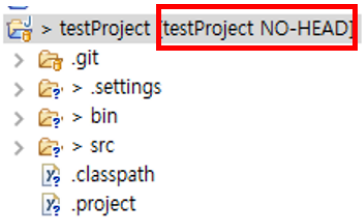
다음으로 로컬 저장소(본인 컴퓨터의 저장 경로) 의 경로를 설정하고 원격 저장소의 호칭을 정하고 Finish를 누르면 Git Repositories 라는 views에 하단의 이미지와 같이 경로가 연결됨을 확인할 수 있다.

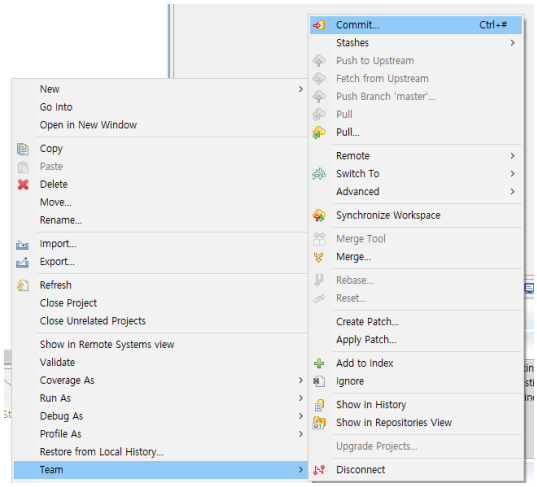


이제 해당 저장소에 프로젝트 파일을 업로드 해보자. 먼저 작업한 프로젝트 상단에 마우스 우클릭으로 Team > Share Project 에 접근, Git을 선택한 뒤 git 별도의 설정파일(.git)을 생성해야 하므로 git 용도의 별도 저장소를 Create Repository를 클릭한다.

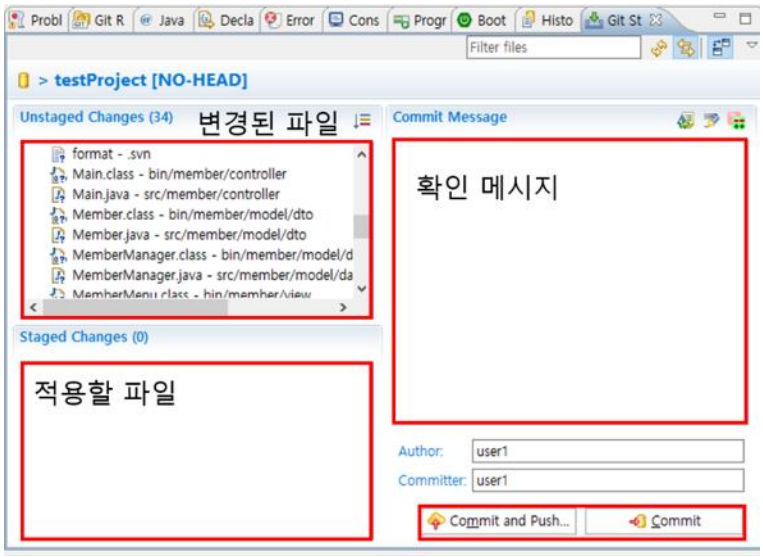


생성이 완료되면 좌측 프로젝트 명 옆에 tag가 붙으며 No-HEAD라고 표시되는데, 이는 아직 최초 commit을 하지 않아 발생한 것이므로, 우 클릭 후 Team > commit으로 소스를 저장해주도록 하자.

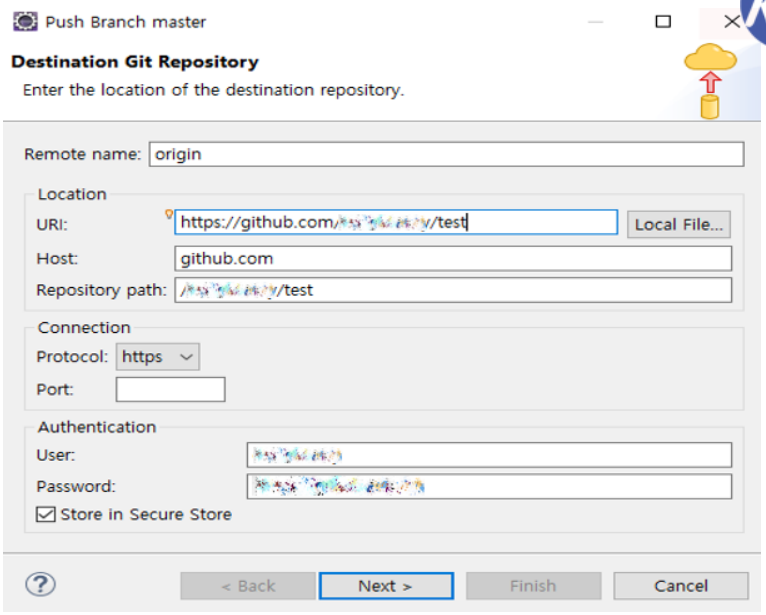




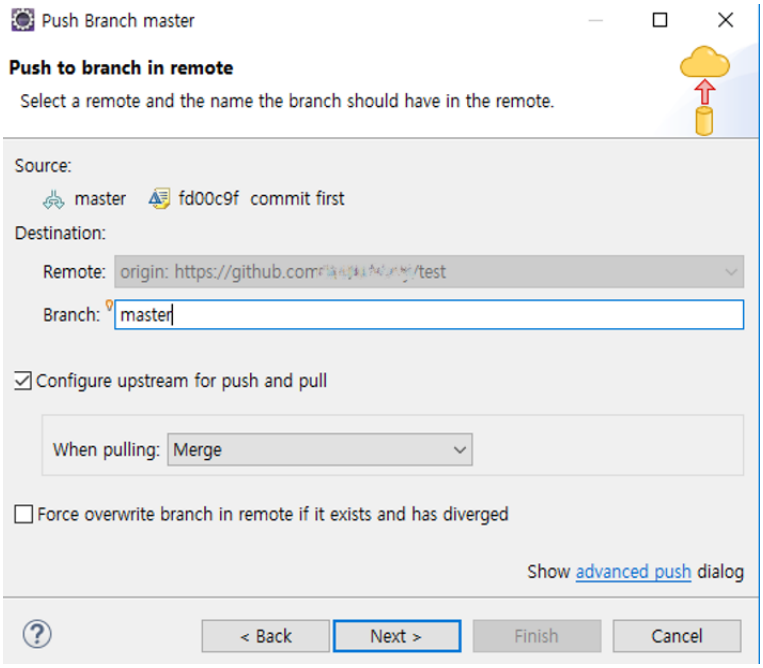
그럼 상단과 같은 창이 하나 발생하는데, 좌측 상단은 변경된 파일, 하단은 저장소에 적용할 파일, 우측 상단은 SVN과 동일한 commit 메시지를 적는 부분, 하단은 각각 저장 후 원격지의 저장소에 반영할 것인지, 단순 저장만 할 것인지를 나타내는 버튼들이 있다.

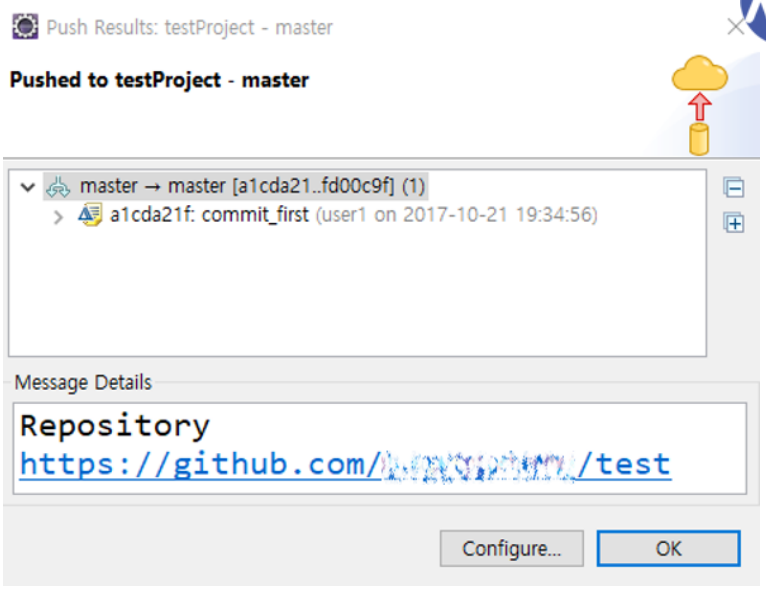


버전에 따라 체크박스 형식으로 파일 선택 창이 나타날 수 있는데, 선택하여 적용하는 방식은 동일하다. 이어서 저장 및 원격지 반영을 하기 위해 Commit and Push 버튼을 클릭하도록 하자. 그럼 목적지 저장소에 대해 설정하는 부분이 나오는데, 이 부분은 처음 github 연동한 것과 동일하게 uri 와 ID, Password를 적어주도록 한다.

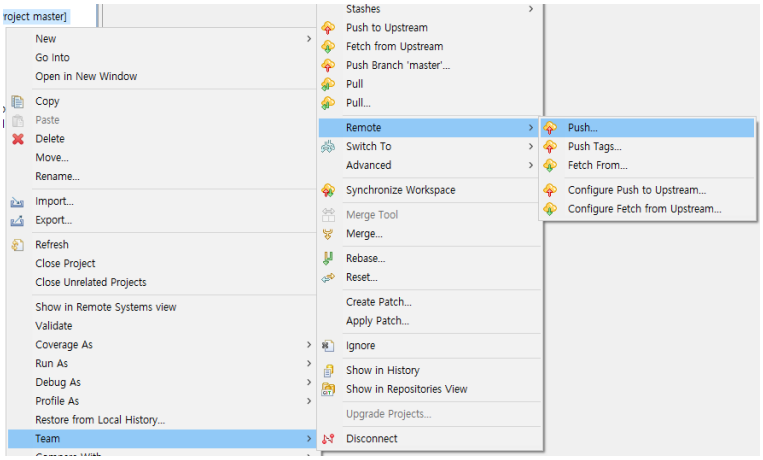


Next를 누르면 우리가 입력한 Commit 메시지와 함께, 반영할 브랜치가 표시되는데, 차 후 팀 프로젝트 작업 시 각자의 브랜치를 생성하여 그 이름을 해당 란에 적으면 된다.

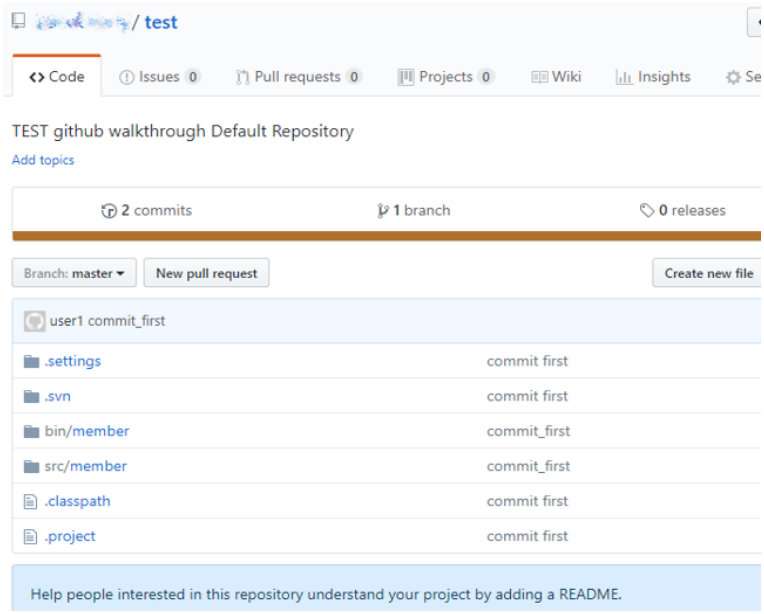




본 이미지와 같이 떴다면, 원격 저장소에 Push가 성공한 것이다. 만약 Reject non-forwarding 에러가 발생했다면, 연동과정에 에러가 발생하여 연동이 제대로 되지 않은 것이므로, Team > Remote > Push 를 통해 진행해보도록 하자. 절차는 commit 과정과 동일하다.

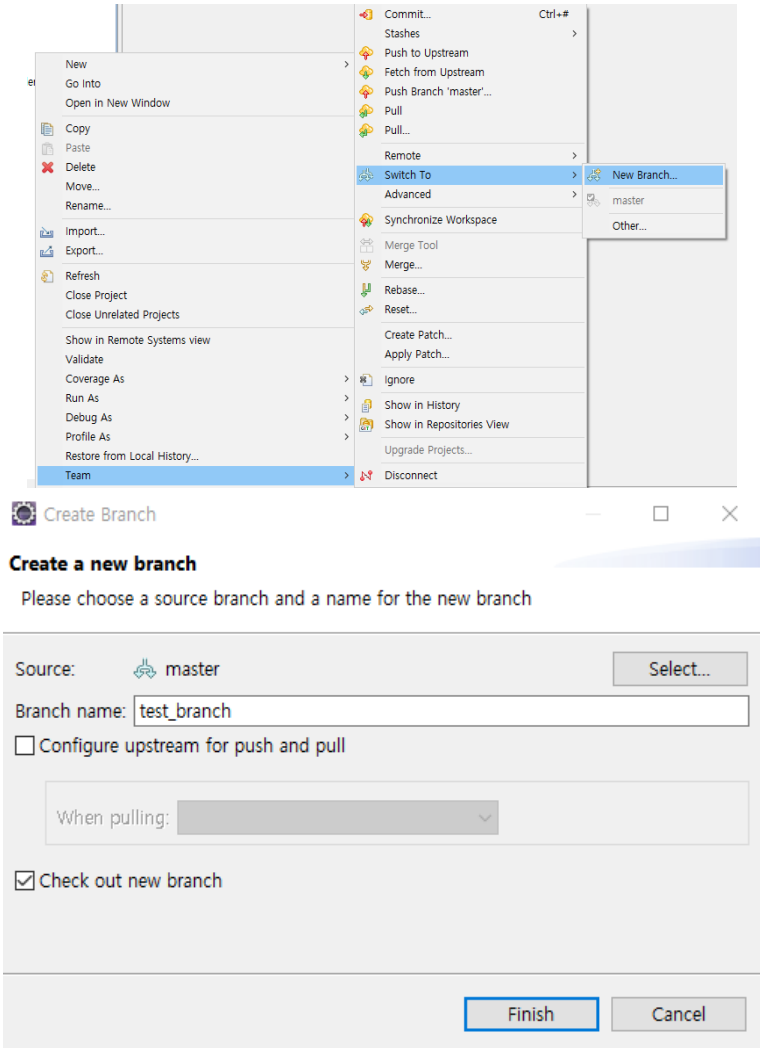


원격 저장소에 반영이 완료되면, github에서도 해당 커밋에 대한 정보를 확인할 수 있다.

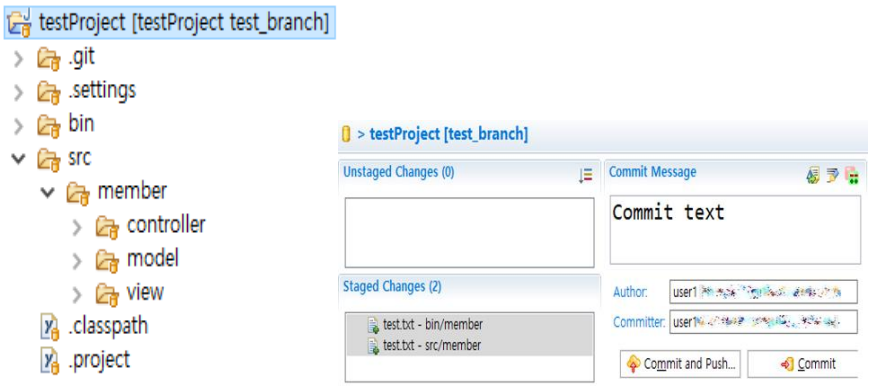


4 Git 소스코드 수정 및 Merge

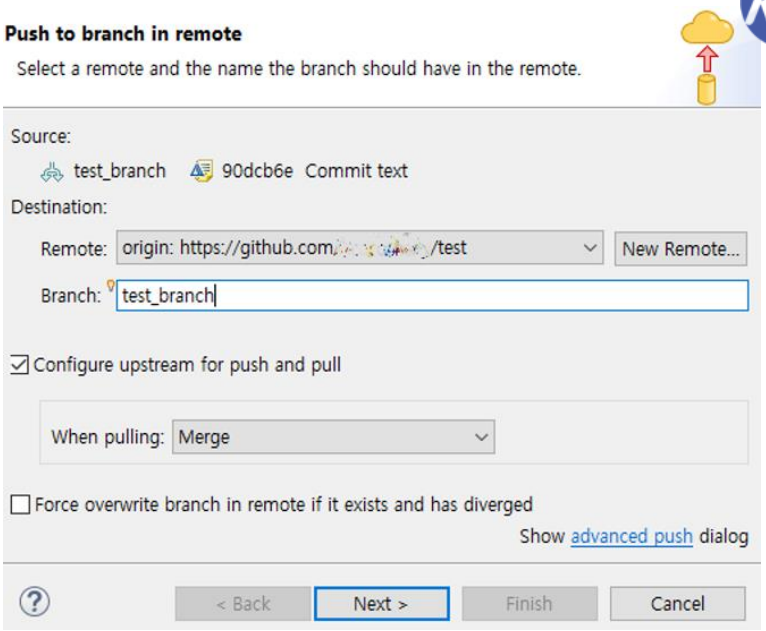
보통 Git에서는 marster에 직접 접근하여 commit 처리를 하지 않고, 별도의 브랜치를 생성하여 작업 후 synchronize 및 merge를 수행한다. 그럼 여기서도 새로 브랜치를 생성하여 작업 내용을 반영해보도록 하겠다. 먼저 Team > Switch To > New branch로 새로운 브랜치를 생성한다.

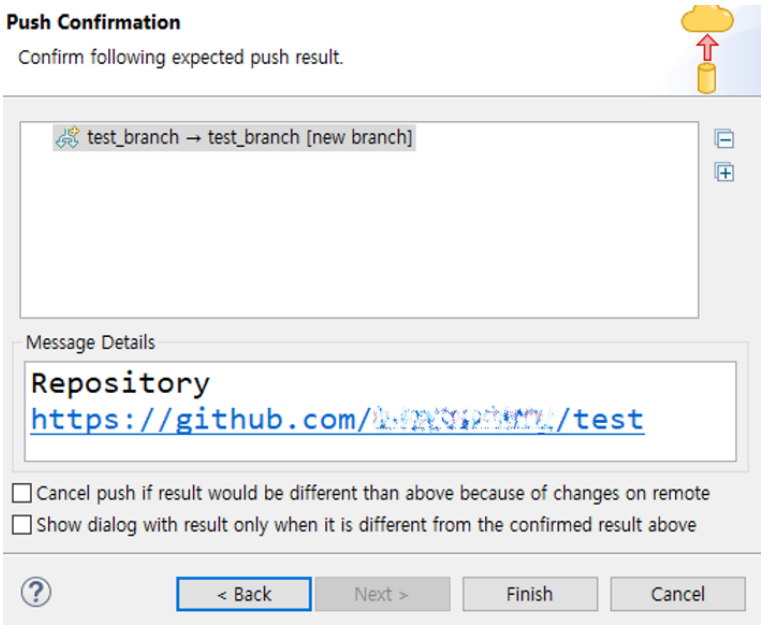


생성을 완료하면 프로젝트 명 우측의 tag로 해당 프로젝트의 포커스가 새로 생성한 브랜치로 잡히는 것을 알 수 있다. 이제 test.txt 파일을 작성하여 커밋&푸시를 수행해보자.

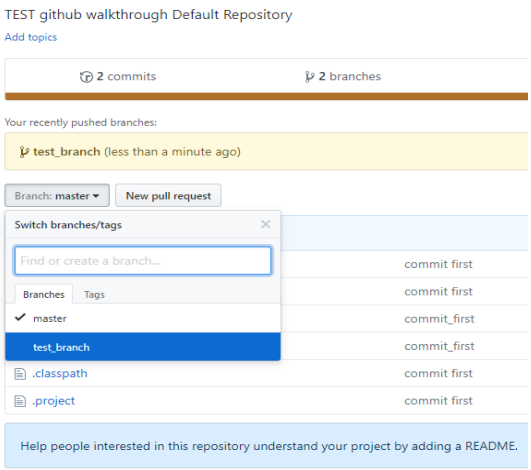


그럼 푸시 대상이 다음과 같이 새로 생성한 브랜치로 잡히는 것을 알 수 있다. 확인 후 Next, Finish를 누른다.



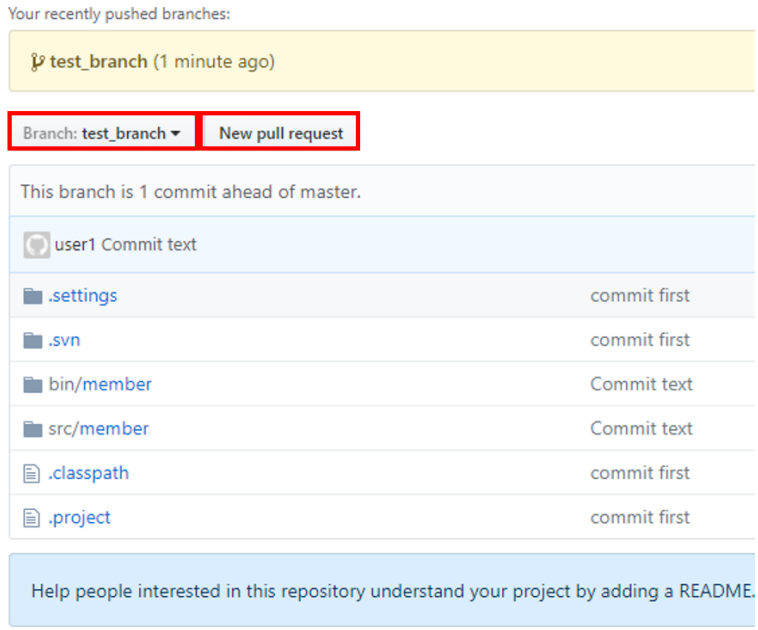


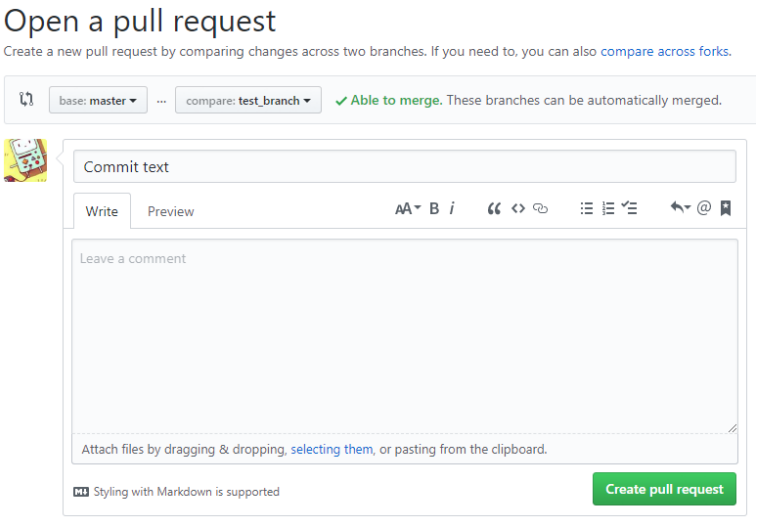
New branch로 생성이 완료되면, 원격 저장소인 Github에서도 확인이 가능하다



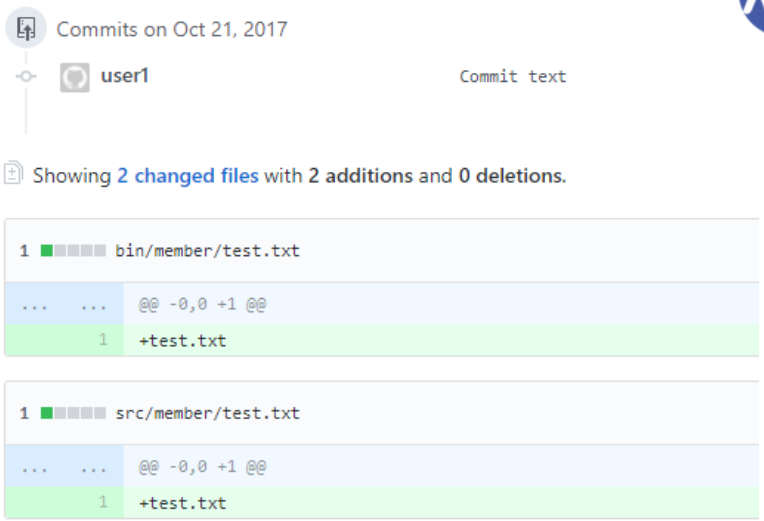
그럼 이제 master에 해당 내용을 반영해보자, git에서는 이 과정을 Pull 또는 Pull Request라고 한다. 다음은 GitHub에서 Pull Request를 수행하는 과정이다. 브랜치를 작업하던 브랜치로 변경한 후, 우측 상단의 new pull

Request를 누르면,

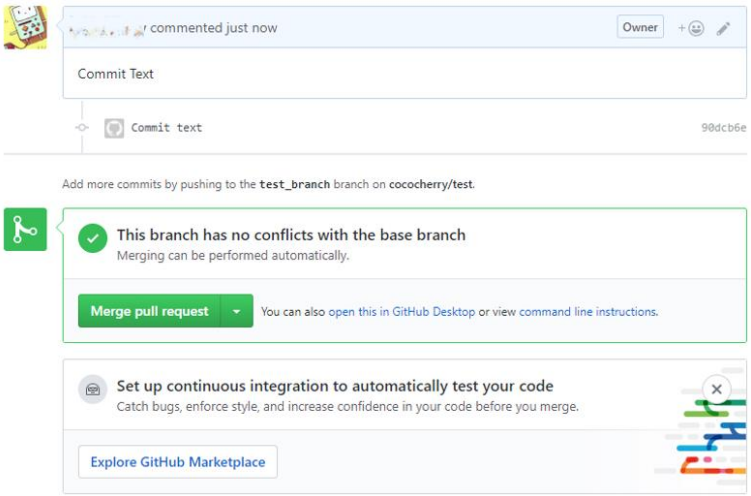




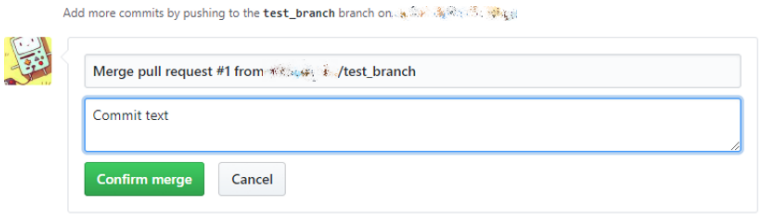
다음과 같은 창이 뜨면서 pull을 요청하는 사유를 적을 수 있으며 상단에는 해당 코드가 충돌이 나지 않는지 검사를 수행하여 가능할 경우 Able to merge라고 표시해준다. 하단에는 변경 사항을 출력해서 보여준다. 만약 충돌이 날 경우 하단에 충돌 나는 사유가 발생하므로 해당 소스를 주석처리 하거나 수정하여 머지를 수행하면 되겠다.

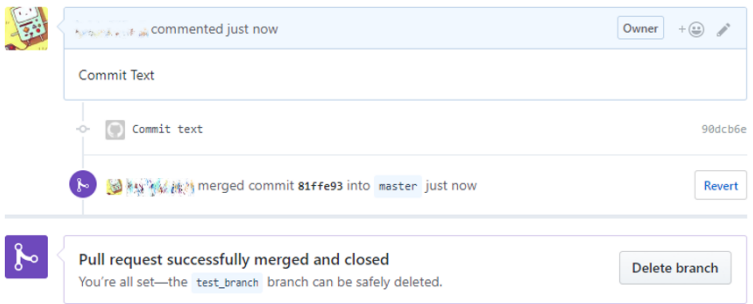


Request를 정상적으로 수행하면 이제 Owner 입장에서는 Merge pull request로 요청받은 머지를 수락할 수 있다.



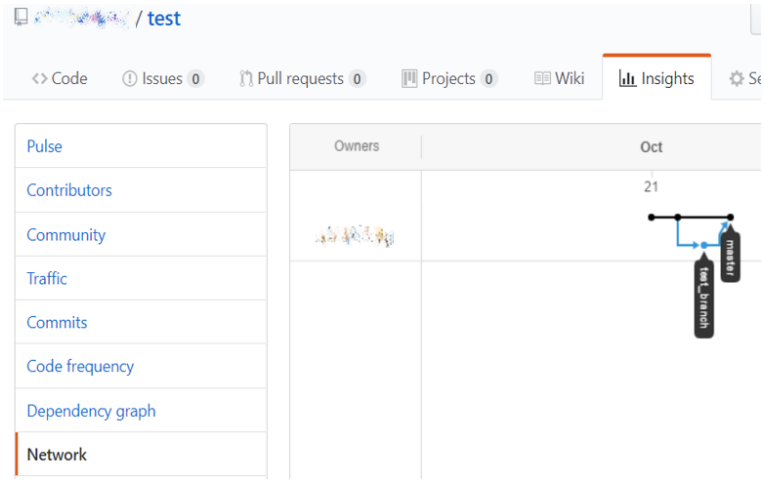
수락하면 확인 메시지를 적을 수 있다.





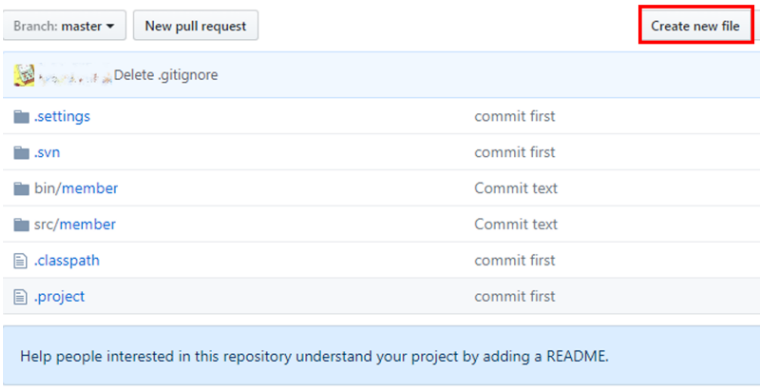
머지가 완료되면, 위와 같은 메시지가 나오고 해당 브랜치를 삭제할 수 있으며, Insights > Network 메뉴에서 다

음과 같이 확인할 수 있다.

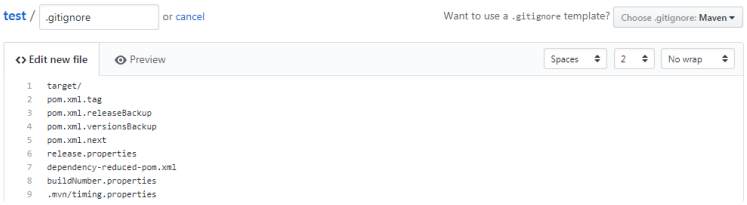


5 Git ignore 설정 활용

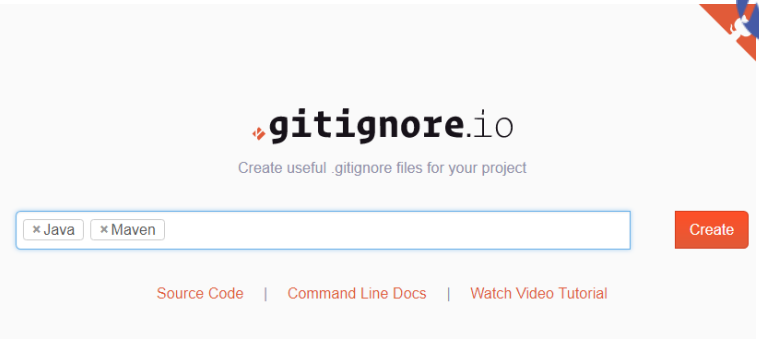
Git Ignore의 통합 설정은 저장소를 생성 시 적용해야 하는데, 그 이유는 중간에 생성하게 되어도 해당 파일이 저장소에 이미 올라와 있을 경우, git의 형상 관리를 받기 때문이다. 따라서 저장소 생성 시 무시하길 원하는 소스에 대한 설정을 하여 ignore 설정을 잡아 주도록 하자. ignore 파일 생성은 프로젝트 최 상단에서 하도록 한다. 먼저 github에서 우측 상단에 Create new file로 새로운 파일을 하나 생성한다.



파일 이름은 .gitignore 로 짓고, 내용 란에, 무시하길 원하는 파일을 적어 주면 된다.
참고로 github에서 제공하는 우측의 양식을 이용해도 된다.



만약 어떤 설정을 잡아줘야 할지 모르겠다면 https://www.gitignore.io/ 사이트를 참조하여 설정을 작성하도록 하자.



gitignore.io는 원하는 키워드를 적은 뒤, Create를 누르게 되면, 밑의 이미지와 같이 필요한 설정에 대한 소스 코드를 작성해준다.

```
# Created by https://www.gitignore.io/api/java,maven

### Java ###
# Compiled class file
*.class

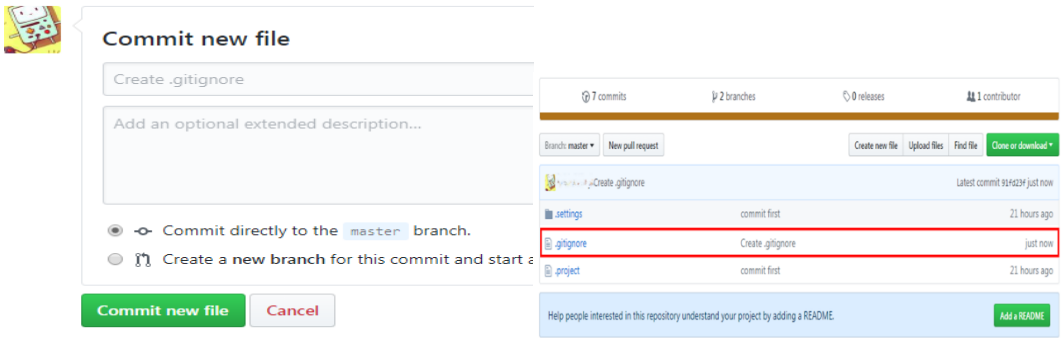
# Log file
*.log

# BlueJ files
*.ctxt

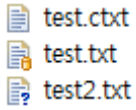
# Mobile Tools for Java (J2ME)
.mtj.tmp/

# Package Files #
*.jar
*.war
*.ear
*.zip
*.tar.gz
*.rar
```

작성이 끝나면 커밋 처리를 하도록 하자. 그 후 화면과 같이 ignore 파일이 생성되어 있으면 성공!



이후 프로젝트를 Check Out으로 불러와서, ignore된 설정이 정상적으로 반영이 되어 있다면 commit 시에 무시할 파일은 목록에 뜨지 않을 것이다.



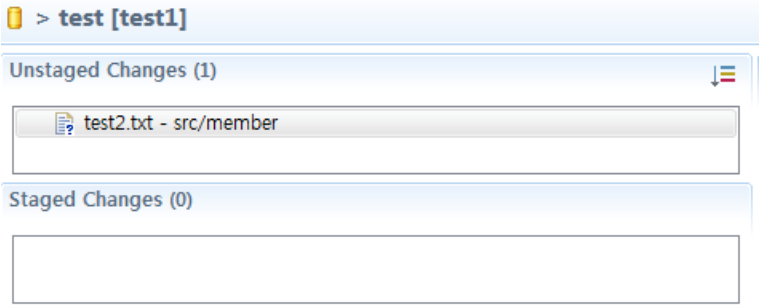
현재 test2.txt와 test.ctxt 두 개를 작성 후 commit 처리하려고 하였을 때, text.ctxt는 정상적으로 무시되어 목록에 없는 것을 확인 할 수 있다.

```
# Created by https://www.gitignore.io/api/java,maven

### Java ###
# Compiled class file
*.class

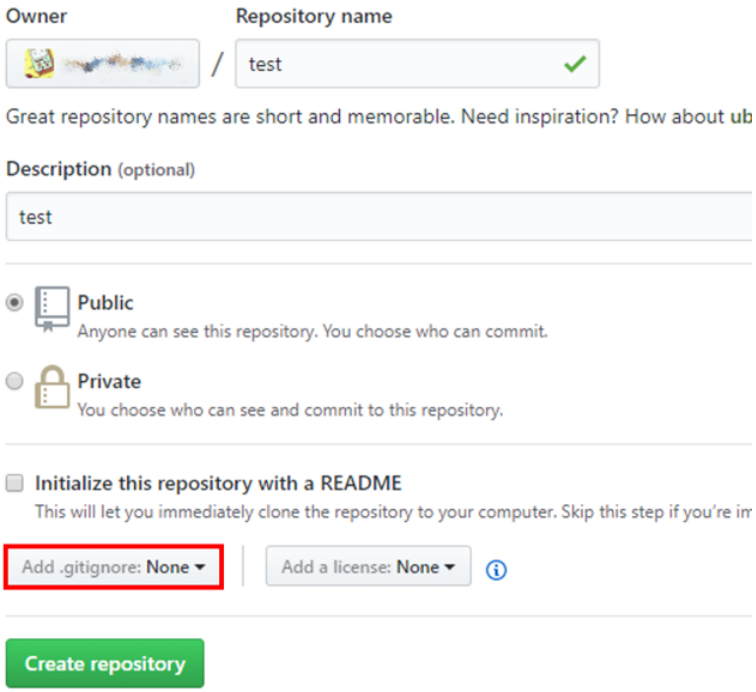
# Log file
*.log

# BlueJ files
*.ctxt
```



Git ignore 설정은 저장소를 생성 시 함께 작성할 수 있는데,

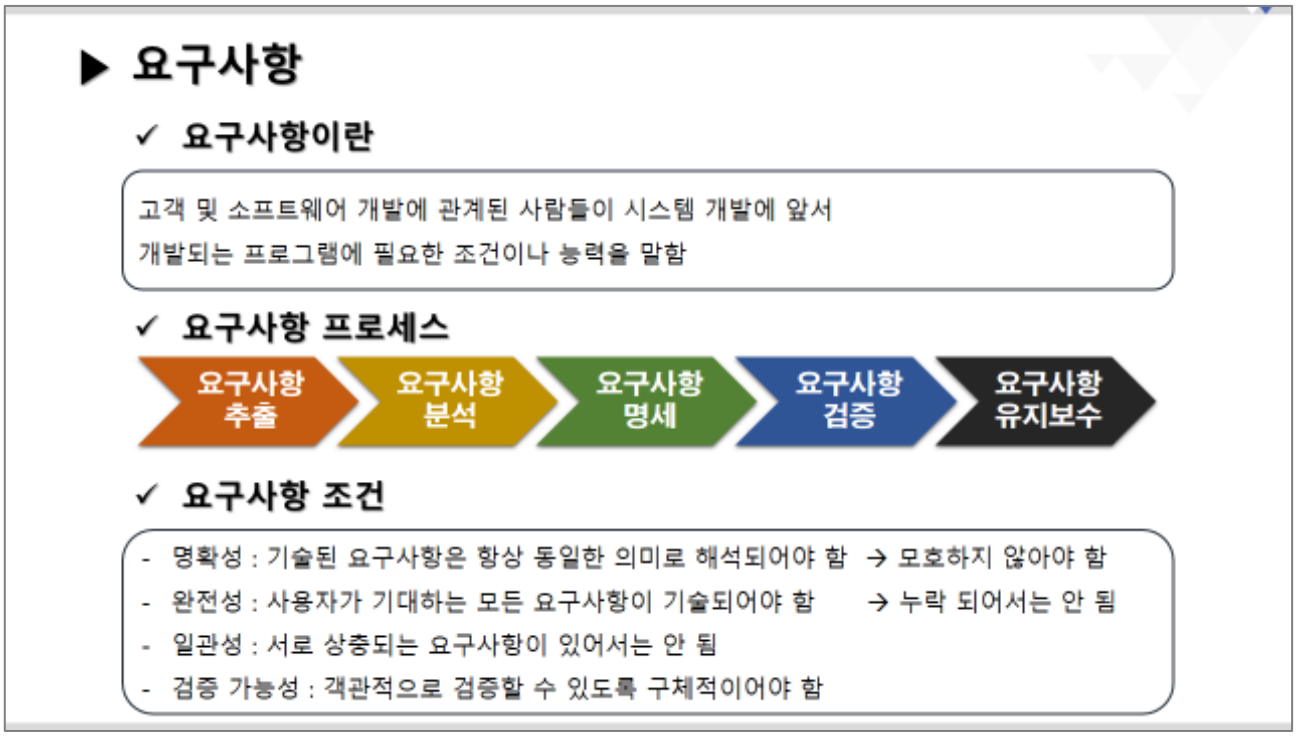
github에서 저장소를 생성 시, 하위 항목을 설정하여 진행하면 된다.



Part 2. 요구사항 확인하기

Chapter 4. 웹 애플리케이션 기능 요구사항 분석

요구사항 개념 및 유스케이스 다이어그램



*** 요구사항 프로세스**

1. **추출 단계 :** 이해관계자 식별, 요구사항 파악 및 수집 (인터뷰, 회의 질문, 의사소통)
2. **분석 단계 :** 요구사항들 간 상충되는 부분 해결, 소프트웨어 환경과 요구사항 도출
3. **명세 단계 :** 요구사항 분석을 기반으로 시스템, 소프트웨어 정의 및 문서로 작성
4. **검증 단계 :** 요구사항의 일관성 및 완전성 등등을 검증, 요구사항 문서 검토

*** 요구사항 조건 예시**

- **명확성 : '학생은 총 10권까지 대출 할 수 있다. '**
  → 반납된 도서 수는 제외된다.
  → 대출 도서 수는 반납과 관계없이 누적된다.
- **완전성 : '주문한 도서가 도착하면 주문자에게 이를 통보한다. '**
  → 주문 후에 일정 기간(예, 1달)이 지나도 주문 도서가 도착하지 않으면?
- **일관성**
  : '사용자는 시스템의 모든 기능을 이용하기에 앞서 시스템에 로그인을 해야 한다.'
  : '도서 검색은 로그인을 하지 않고도 가능하다.'

- **검증 가능성 : '도서 검색은 최대한 빨리 수행되어야 한다. '**

  → "빠른"의 기준은 무엇인가?

  → 검색 범위에 관계없이 동일한 성능을 제공해야 하는가?

Chapter 5. 요구사항 정의 명세서 작성

1. 요구사항 정의서

- 요구사항들의 목록화를 통해 전체적인 요구사항의 리스트를 확인 할 수 있음
- 요구사항들의 리스트를 확인하는 문서
- 자세한 내용은 명세서를 확인

▶ 요구사항 정의서 구성요소

1) 프로젝트 명, 작성자, 작성일
2) 요구사항분류 (업무 영역)
   ex) 업무 : 신규 기능 추가 > 요구사항 : 주요한 메뉴 퀵메뉴 사용
   ① 기능 요구사항
   ② 성능 요구사항
   ③인터페이스 요구사항
   ④운영 요구사항
   ⑤ 자원 요구사항
   ⑥검증 요구사항
   ⑦ 테스트 요구사항
   ⑧ 문서화 (산출물) 요구사항
   ⑨ 보안 요구사항
   ⑩ 품질 요구사항
   ⑪ 유지보수 요구사항
   ⑫ 웹접근성 / 웹표준 요구사항
   ⑬ 기타
3) 요구사항 ID
   ex) R-001
   : 요구사항 ID 는 모든 산출물 문서에 표시해야 함
   : 화면 설계서에서도 요구사항 ID 가 있어야함
   : 요구사항 추적표에도 있어야 함

2. 요구사항 명세서

: 요구사항들의 상세 내역들을 확인할 수 있습니다.

▶ 요구사항 구성요소

1) 업무 영역(요구사항 분류 : 요구사항정의서의 요구사항분류 표시)
   ex) 신규기능 추가

2) 요구사항 ID
   ex) R-001

3) 요구사항 명
ex) 주요한 메큐 퀵메뉴 사용

4) 개요
ex) 중요한 메뉴들은 모든 페이지에서 볼 수 있도록 퀵메뉴 사용합니다.

5) 상세설명
ex) 전자민원, 공인인증서, 전자민원 미리해보기, 인원현황 조회는 중요한 부분으로
모든 페이지에서 확인 할 수 있었으면 한다.

6) 유형 (요구사항분류)
ex) 기능

7) 중요도
ex) 중

8) 난이도 (작업의 난이도)
ex) 중
9) 출처 (회의록 FGI/UT, 제안서)

10) 관련부서 (관련 담당자)
ex) 포탈 서비스 팀

3. 주의 사항

1) 요구사장 정의서 & 명세서 = 프로젝트의 **분석단계에서 가장 중요한 산출물**
Because 프로젝트의 **검수 단계에서도 가시 나타나게 되기 때문**

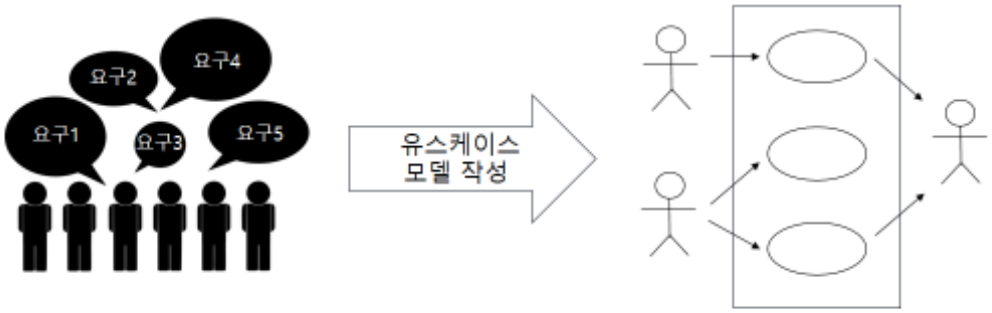
2) 프로젝트가 끝나는 시점까지 요구사항은 늘어날 수 있어 각 문서들은 버전관리 (문서 이력)을 관리해야함

3) 무엇보다 이런 요구사항들이 모두 적용 되었는지 추적표를 작성하여 관리자는 항시 체크를 해야 함

▶ 유스케이스 다이어그램

✓ 유스케이스 다이어그램

동적(행위) 다이어그램으로 시스템 내의 활동들의 흐름을 보여줌
여러 업무 프로세스를 설명하는데 자주 활용

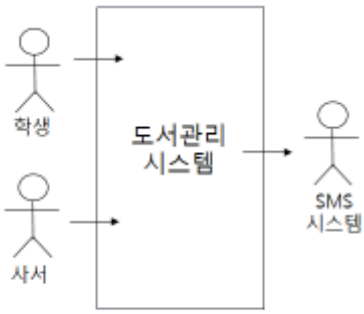


▶ 유스케이스 다이어그램

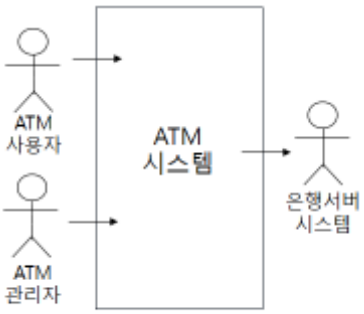
✓ 액터

시스템과 상호작용을 하는 시스템 외부의 존재로 개발 대상에 따라 달라질 수 있음
시스템 관점에서 바라본 사용자의 역할을 뜻해야 함

ex) 도서관리 시스템의 액터



ex) ATM 시스템의 액터

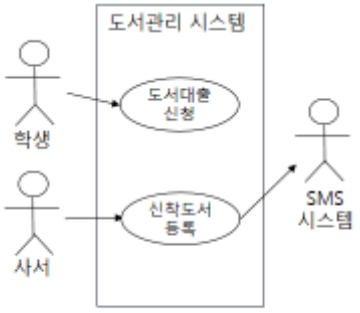


▶ 유스케이스 다이어그램

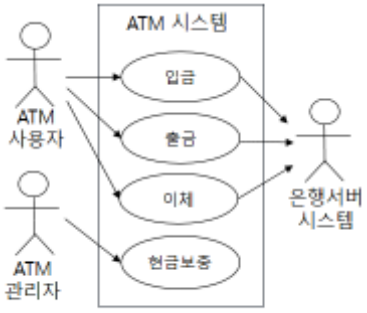
✓ 유스케이스

개발 대상이 되는 시스템이 제공하는 개별적인 기능을 뜻하는 것으로 시스템 동작 하나의 기술, 사용자가 인지할 수 있는(눈에 보이는) 하나의 기능 단위

ex) 도서관리 시스템의 유스케이스

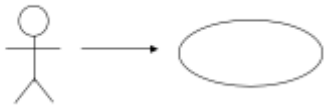


ex) ATM 시스템의 유스케이스



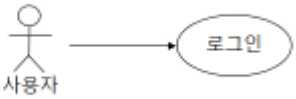
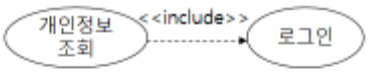
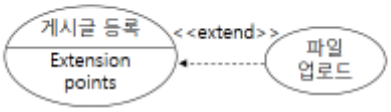
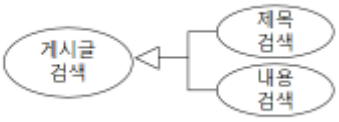
▶ 유스케이스 다이어그램

✓ 액터와 유스케이스 간의 연관 관계 방향

유형	설명	연관 관계 방향
활성화	액터가 유스케이스를 활성화 시킴	
수행결과 통보	유스케이스 결과가 액터에게 통보 됨	
외부 서비스 요청	외부 시스템에 서비스 실행을 요청함	

▶ 유스케이스 다이어그램

✓ 유스케이스 다이어그램 관계 종류

유형	설명	관계 방향
연관 관계	유스케이스와 액터 간 상호작용을 의미하는 관계	
포함 관계	한 유스케이스가 다른 유스케이스의 기능을 포함하는 관계 (반드시 해야만 하는 관계)	
확장 관계	기본 유스케이스에서 특정 조건이나 액터의 선택에 따라 발생하는 유스케이스 (선택적으로 할 수 있는 관계)	
일반화 관계	유사한 유스케이스들 또는 액터들을 추상화한 하나의 유스케이스로 그룹핑하여 이해도를 높인 관계	

*** 의존관계(점선)**

포함관계 : 여러 유스케이스에서 공통적인 기능의 표현(재사용의 의미) → 의존함

확장관계 : 한 유스케이스에서 부가적인 부분 기능의 표현 → 의존하지 않음

*** <<include>> 포함관계의 부가설명**

- **포함관계를 만드는 것은 '기본 유스케이스'의 시나리오가 진행되면서 '포함 유스케이스'의 시나리오는 '반드시' 필요로 할 때 성립한다.**

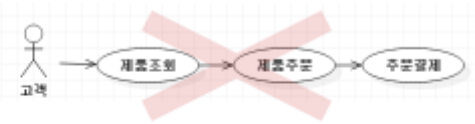
  → 포함 유스케이스가 실행되지 않는다면 기본 유스케이스가 본연의 기능을 발휘하지 못하는 것을 의미

- **포함관계로 나타낸 '포함 유스케이스'가 다른 유스케이스에서 사용된다면 더욱 가치 있는 것이 된다.**

  → 포함관계로 포함 유스케이스를 따로 만드는 큰 이유를 '재사용 ' 에 두고 있다.

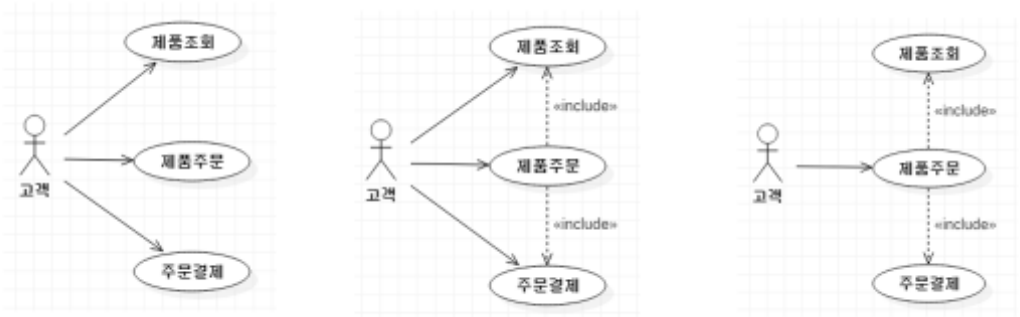
▶ 유스케이스 다이어그램 상황별 예시

✓ 시나리오상 다음과 같은 흐름 인식



연관관계(실선)는 액터와 유스케이스 간만 사용 가능

✓ 의미에 따른 모델링



*** 유스케이스 다이어그램을 작성할 때 시나리오 흐름 즉 시퀀스 구조를 생각하면서 그리는 사람들이 존재**

→ 이는 '제품주문'이라는 행위를 하기 위한 '제품조회 ' 라는 사전조건과 '주문결제 ' 라는 사후조건을 생각한 것이다.

1. 연관관계(실선)는 액터와 유스케이스 사이에서만 사용 가능하다. 유스케이스와 유스케이스 사이에서는 사용할 수 없다.

2. 의존관계(점선)는 유스케이스와 유스케이스 사이에서만 사용할 수 있다.

3. 일반화 관계는 액터와 액터 사이, 유스케이스와 유스케이스 사이에서만 사용 가능하다.

>> 위의 그림1~3 모두 어디에 중점적으로 의미를 두느냐에 따라 모델은 항상 달라진다.

그림1 : 유스케이스 다이어그램을 작성할 때 왼쪽 위에서 오른쪽 아래방향으로 시간적인 순서가 되도록 작성하는 가이드에 맞춰서 표현한 예

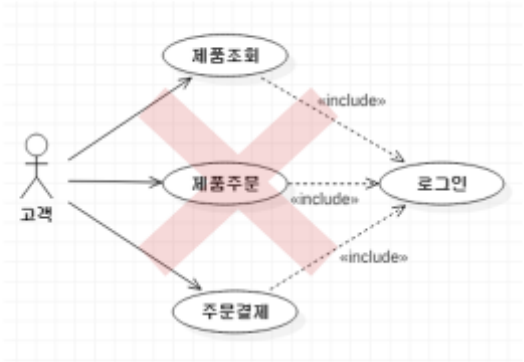
그림2 : 그림1에서 <<include>> 포함관계를 같이 표시한 예

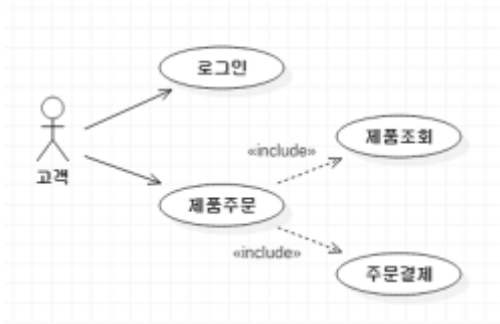
그림3 : 그림2가 너무 복잡해서 활성화 부분은 뺀 내용

답이 없다! 다만 include를 쓰느냐 마느냐의 문제는 재사용성에 중점을 두고 모델링을 하는 것이 좋다.

▶ 유스케이스 다이어그램 상황별 예시

✓ 로그인 유스케이스





*** '로그인'은 과연 포함 유스케이스일까??**

그림1.

→ 아마도 '로그인' 유스케이스는 다른 유스케이스 시나리오의 첫번째 부분을 담당

: 다른 유스케이스들의 '사전조건'이다. (인증과 같은 의미)

→ 하지만 모든 유스케이스 마다 로그인 유스케이스를 포함관계로 만들게 되면 아주 복잡한 모델링이 될 것이다.

그림2.

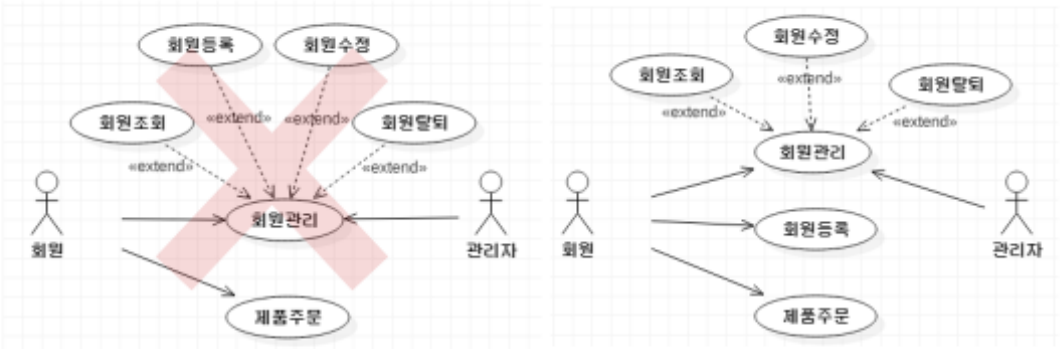
→ 사전조건 역할의 유스케이스는 포함관계보다 독립적인 유스케이스로 정의하는게 가독성을 높이고 관리가 용이하다.

→ 독립적인 유스케이스로 둬도 개발자나 사용자들도 서로가 로그인을 해야만 다른 서비스를 할 수 있다는 건 기정 사실처럼 알기 때문에 복잡하게 그리지 말자는 것!

→ 액터의 입장에서 요구하는 진정한 서비스는 로그인 보다 그 외의 유스케이스이다!! 거기에 포커스를 맞추자!!

▶ 유스케이스 다이어그램 상황별 예시

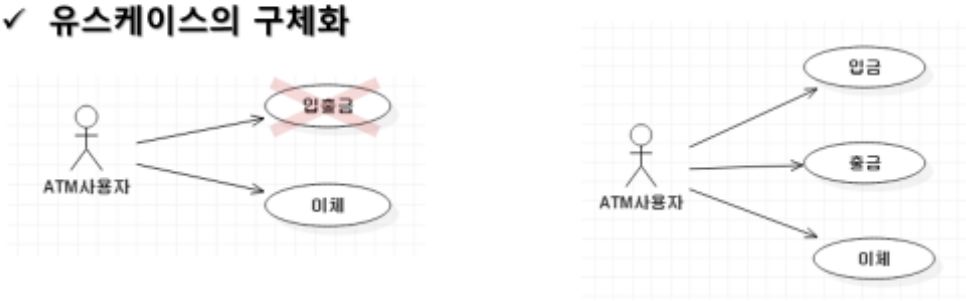
✓ 유스케이스의 동일한 기능 제공



* 회원관리라는 한 유스케이스가 등록,조회,수정,탈퇴의 의미를 가질 때 이 기능을 활성화 하는 액터인 회원, 관리자가 동일한 기능을 가져야 되는데 회원등록과 같은 기능은 회원만 해당되는 기능이기 때문에 다음과 같이 회원등록을 독립적인 유스케이스로 뽑아내야 된다.

▶ 유스케이스 다이어그램 상황별 예시

✓ 유스케이스의 구체화

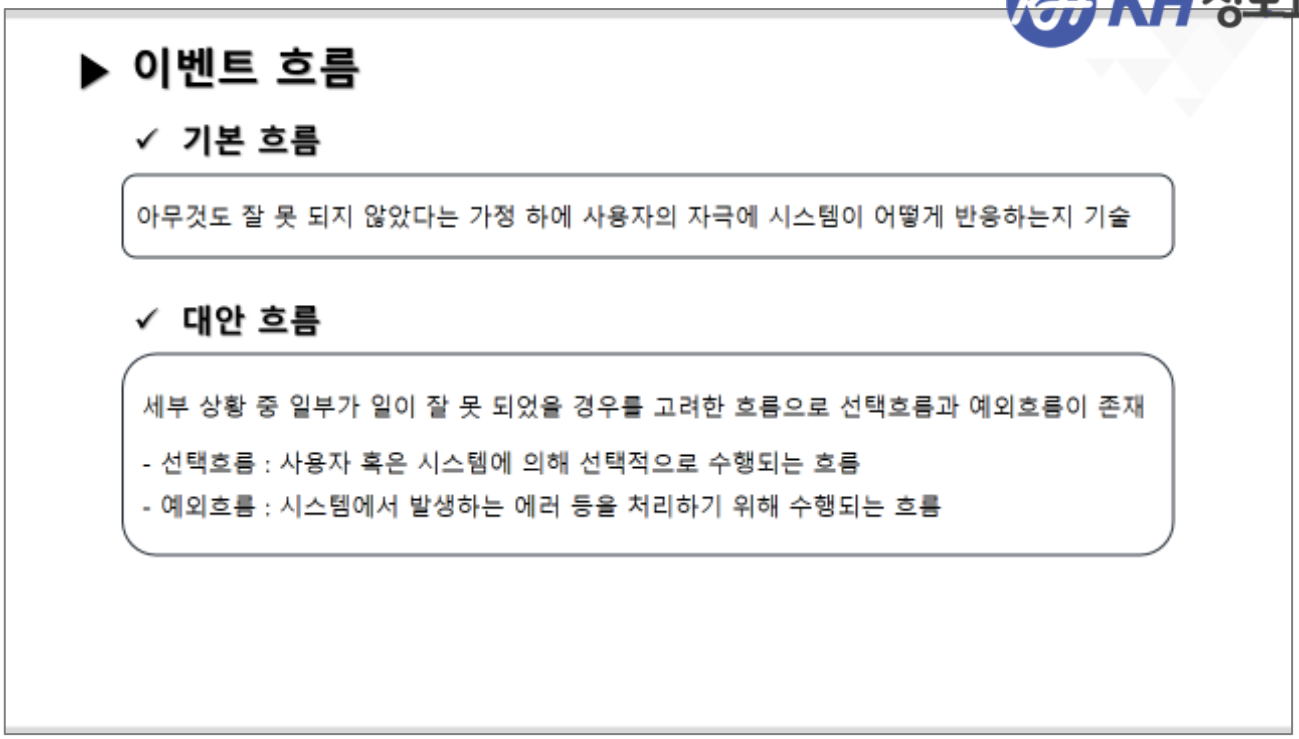


✓ 다양한 세부상황 표현



* 유스케이스는 구체적이어야되고 사용자가 인지 할 수 있는 하나의 기능 단위여야 된다.

* 하나의 독립적인 기능을 구성하는 다양한 세부 상황은 하나의 유스케이스로 표현되어야 된다.



*** 다양한 세부상황, 즉 기본흐름 외에 선택흐름 및 예외흐름이라는게 존재하는데 이를 유스케이스에서 하나의 유스케이스로 표현되어야 된다고 했다. 그럼 이러한 이벤트 흐름(선택, 예외 흐름)은 어떻게 모델링을 하느냐??**

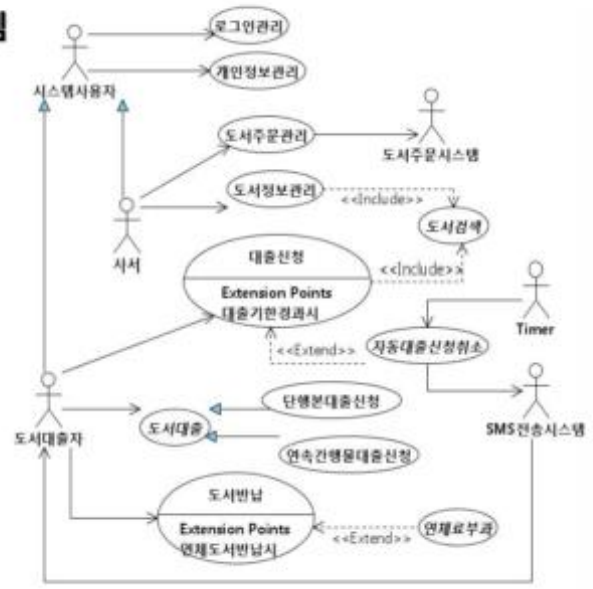
➔ 유스케이스 다이어그램 작업이 끝나게 되면 유스케이스 시나리오라는 걸 쓰게 된다.

*** 유스케이스 시나리오가 포함할 것 :** 유스케이스를 수행하는 행위자, 선행 및 후행조건, 유스케이스의 이벤트 흐름

➔ 이러한 유스케이스 시나리오를 바탕으로 속성, 오퍼레이션, 관계를 도출하여 클래스, 시퀀스 다이어그램 작업을 하게 될 것이다.

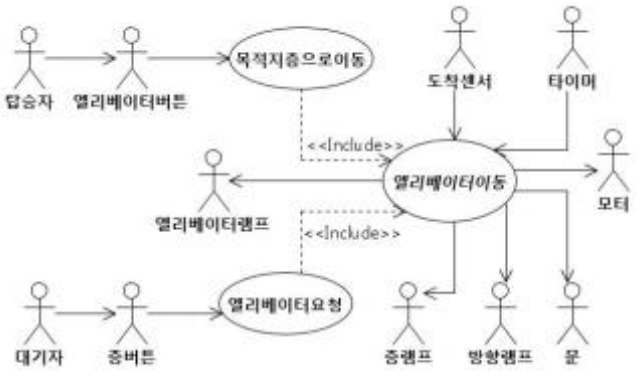
▶ 유스케이스 다이어그램 예시

✓ 도서관리 시스템



▶ 유스케이스 다이어그램 예시

✓ 엘리베이터 시스템



< 실습 >

1. 다음과 같은 요구사항 기술서 배포

인터넷 쇼핑몰 시스템은 상품의 판매나 구입을 간편하게 하기 위한 시스템이다.

해당 인터넷 쇼핑몰 사용자는 로그인을 통해 쇼핑몰을 이용할 수 있다. 따라서 쇼핑몰을 이용하기 위해 일반 고객은 회원가입을 할 수 있다.

사용자는 마이페이지를 통해 자신의 정보를 조회할 수 있고, 이 때 수정 또는 탈퇴를 할 수 있다. 관리자는 모든 회원들의 정보를 조회할 수 있으며 회원들의 요청에 따라 회원 탈퇴 또는 회원 수정을 할 수 있다. 뿐만 아니라 상품 관리 (등록, 수정, 삭제), 통계조회 (방문자수 통계조회, 상품별 판매 통계조회) 등과 같은 활동을 수행한다.

상품목록에 대한 조회는 고객뿐만 아니라 관리자도 이용가능하며 상품목록조회를 통해 보여지는 전체 목록 중 검색하고자 하는 상품이 있다면 상품검색을 통해 조회할 수도 있다. 상품 상세 조회를 원한다면 보여지는 상품 목록들 중 해당 상품을 선택하면 된다.

고객이 상품을 구매하기 위해서는 반드시 상품 상세조회를 통해 수행해야 되며 결제 완료 시 sms시스템의 문자서비스를 통해 고객과 관리자에서 문자 전송이 이루어진다.

고객은 다른 고객들이 작성한 리뷰를 조회할 수 있고 검색을 통해서도 조회가 가능하다. 상품 구매자에 한해서 리뷰 등록을 할 수 있고 리뷰 등록 시 사진등록을 할 수도 있다.

2. 위의 기술서에서 액터 및 유스케이스 도출

3. 툴을 이용하여 액터와 유스케이스 무작위 배치

4. 중복 의미 유스케이스 제거 및 그룹핑 작업

5. 관계 설정 후 유스케이스 다이어그램 완료!

*** 모범 답안 및 세부 내용은 별로 자료를 참고해주세요**

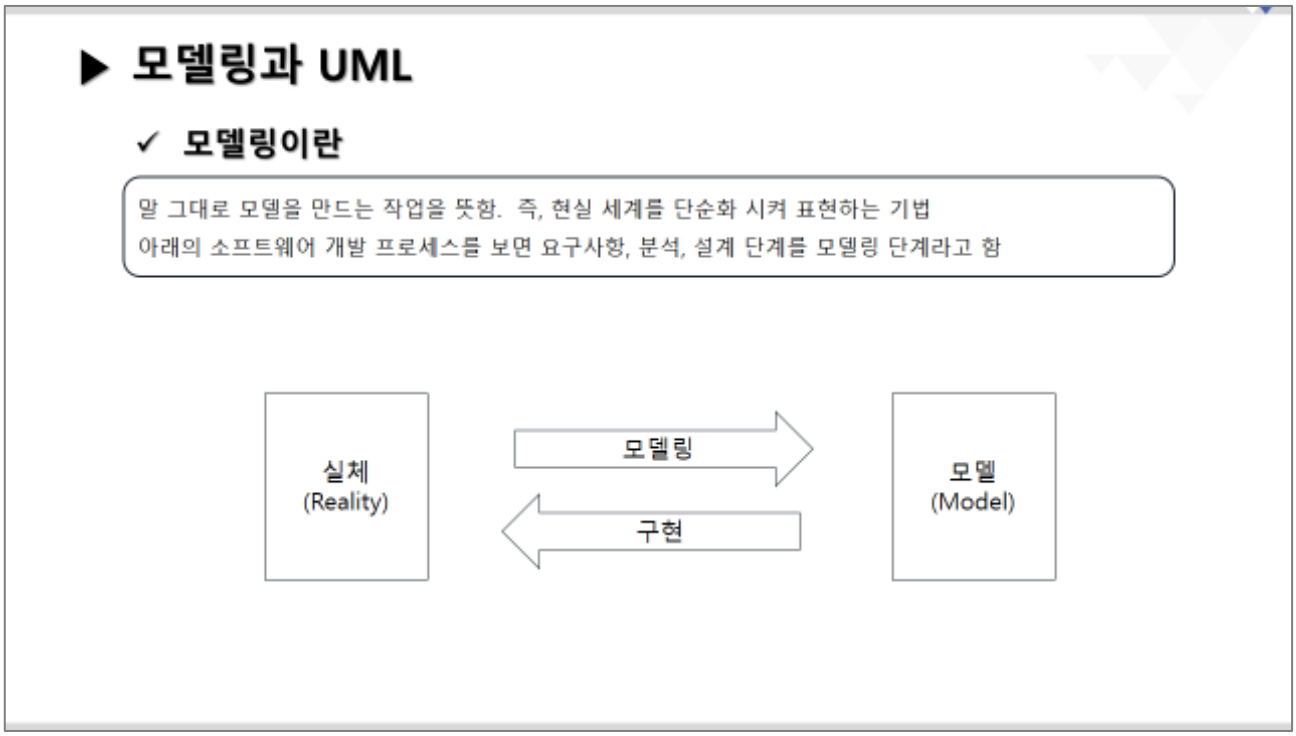
이렇게 유스케이스 다이어그램 작업이 끝나게 되면 실제로 한 유스케이스(기능)별로 시나리오가 나와야 된다고 했다!! 실제로 회사별로 시나리오는 엄청 다양하고 쓰는 방식들이 다르다. 샘플들을 한번 봐보자~~!!

우리는 시나리오 쓰는 작업은 생략하겠다(시간관계상).. 대신 내가 써놓은 시나리오를 배포할 테니 그걸 기반으로 클래스랑 시퀀스 작업을 해보자

Part 3. 분석모델 확인하기

Chapter 7. 기능별 상세 명세서 작성

모델링의 개념 및 필요성 / uml의 개념 및 종류



*** 객체 지향 모델링 언어**

초기 모델링 기법 → 객체지향 X, 절차지향 O (구조적인 모델링 기법)

→ 제대로 반영이 안됨 → 한계 도달

→ **객체 지향 모델링이라는 새로운 패러다임**

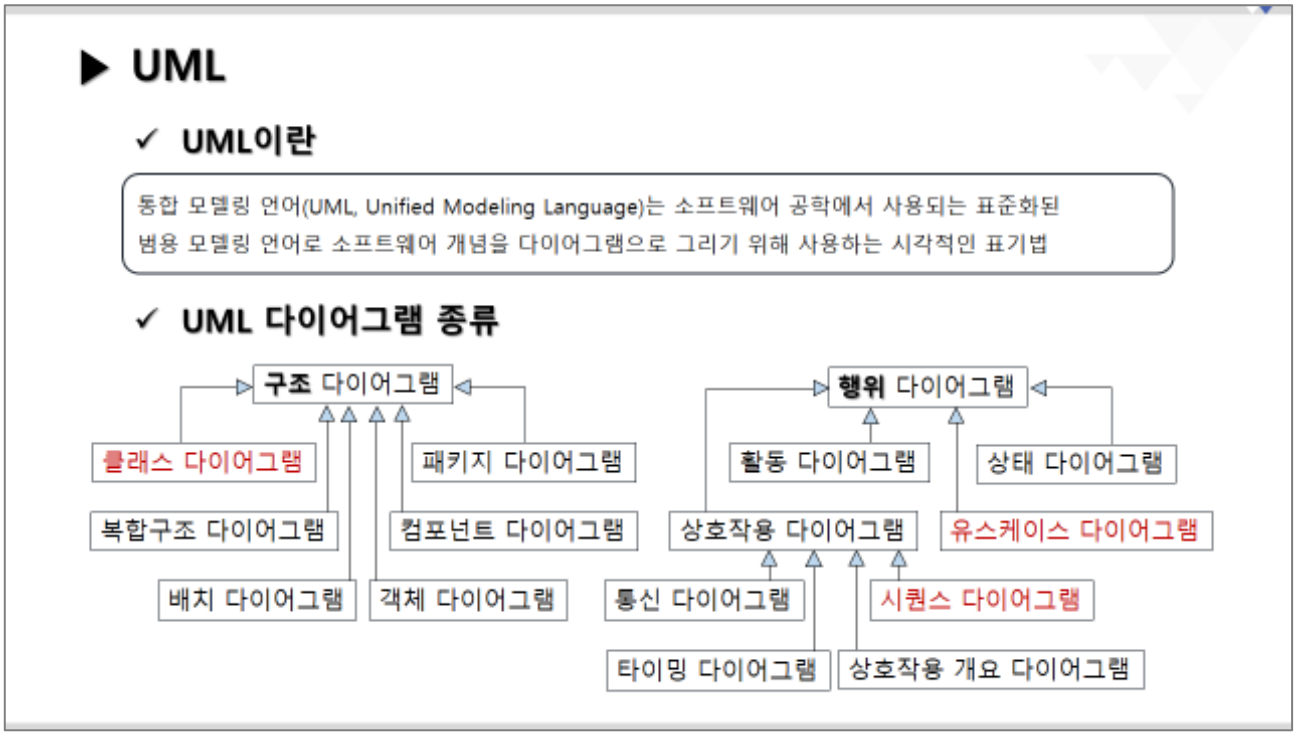
But, 모델링 기법이 다양 → 표기법 또한 다양 → 호환이 안됨

→ OMG에서 UML을 표준 객체 지향 모델링 언어로 공표!

*** 모델링의 필요성**

건축을 짓기 전엔 건축 모델, 비행기를 만들기 전엔 항공 모델, 다리를 만들기 전엔 다리 모델.

→ 사이즈가 큰 것은 기본적으로 모델을 만들고 잘 되는 지 시험을 한 뒤에 작업에 들어간다. 프로그래밍은 건물, 비행기, 다리처럼 사이즈가 크진 않지만 여러 명과 함께 하는 공동 작업이기 때문에 **UML을 통해 의사소통을 활발히 하고 소프트웨어 구조의 재사용 문제를 해결한다.**



*** UML의 필요성**

1. 의사소통 하기 좋다.

- 고객과 개발자들 간에 의사소통을 원할 하게 해줄 뿐만 아니라 요구사항에 부합한 시스템을 개발할 수 있도록 해준다.

2. 대규모 프로젝트 구조의 로드맵을 만들 때 유용하다.

- 클래스와 클래스 간에 의존하는 관계 등등을 개발자가 빨리 파악할 수 있다.

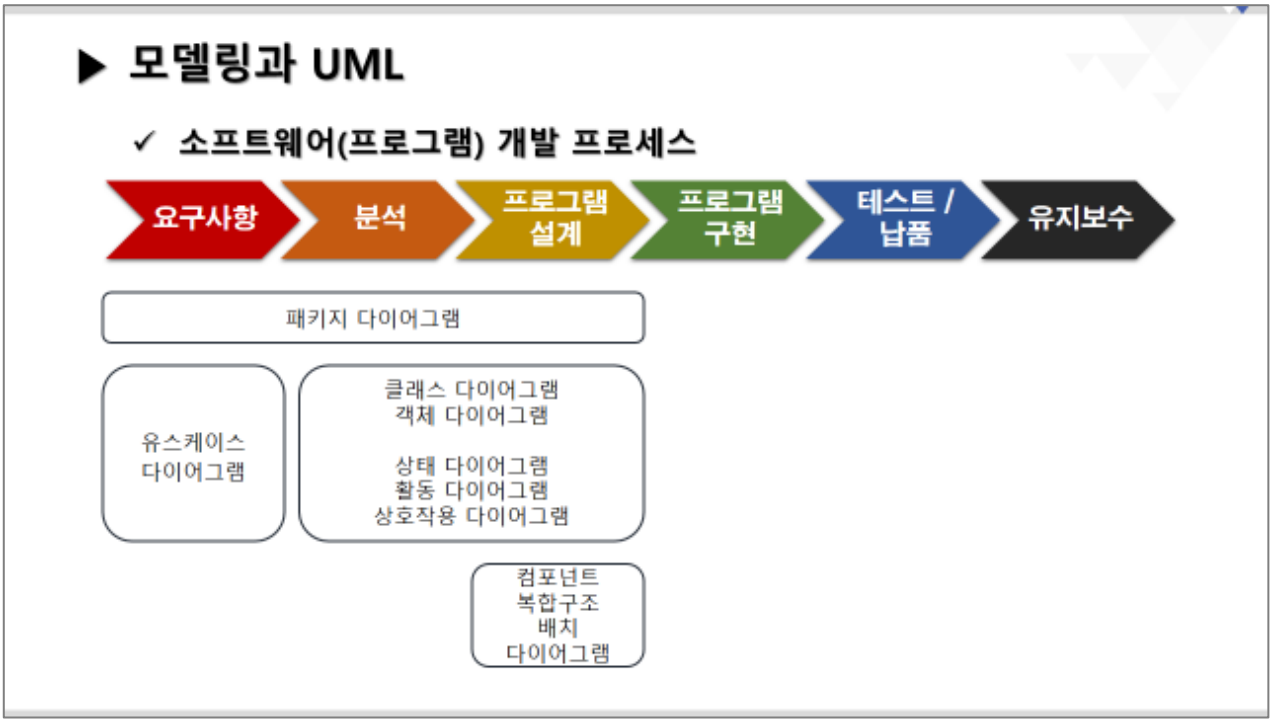
3. 개발할 시스템 구축에 대한 기초를 마련할 수 있다 .

- 모델링 툴에서 소스코드 생성 기능 같은 것들을 제공해준다.

→ 개발자의 입장으로써 로직이나 일부 내용만 수정하면 되기 때문에 구현의 시간을 단축 시킬 수 있다.

4. 백엔드 문서용으로 제격

- 다음에 현 프로젝트를 맡을 팀에게 유용하다.



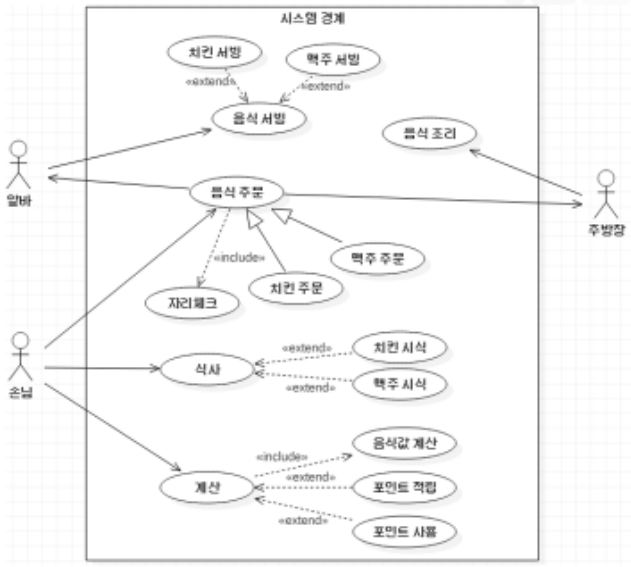
*** uml 다이어그램 종류 및 간략 설명 참조**

분류	다이어그램 유형		목적
정적	클래스		프로그램 안의 주요 클래스와 주요 관계를 보여줌
	객체		시스템 실행 중 어느 순간의 객체와 관계를 포착해서 보여줌
	복합구조		내부 구조를 표현하는 다이어그램
	배치		소프트웨어, 하드웨어, 네트워크를 포함한 실행 시스템의 물리 구조 표현
	컴포넌트		컴포넌트 사이의 의존관계 묘사. 컴포넌트를 구성하는 요소들과 그것들을 구현하는 요소들도 모두 표현 가능
	패키지		대규모 시스템에서 주요 요소간의 종속성을 나타내거나 여러 클래스들의 그룹화된 매커니즘을 나타낼 때 쓰임
동적	활동		플로우 차트가 uml에 접목된 개념, 여러가지 행위들과 제어구조 등을 모두 포함
	상태		한 객체의 상태 변화를 다이어그램으로 표현한 것
	유스케이스		시스템과 사용자가 상호작용하는 경우를 나타내는 기능 위주의 다이어그램
	상호작용	시퀀스	시간 흐름에 따른 객체 사이의 상호작용 표현
		상호작용	여러 상호작용 다이어그램 사이의 제어흐름을 표현

		개요	
		통신	객체 사이의 관계를 중심으로 표현
		타이밍	객체 상태 변화와 시간 제약을 명시적으로 표현

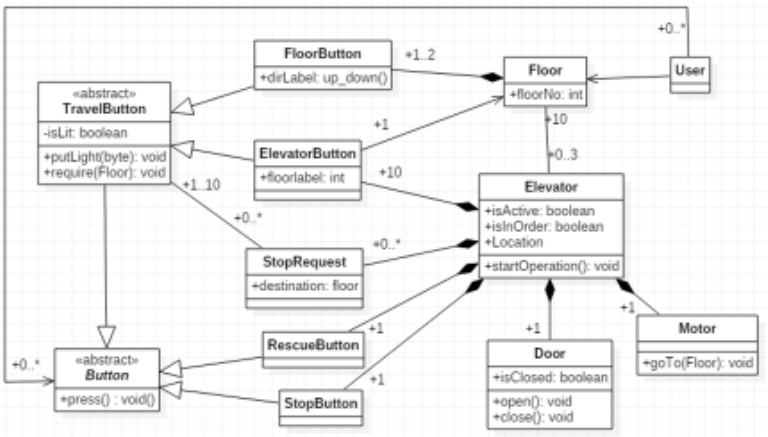
▶ 모델링과 UML

✓ 유스케이스 다이어그램



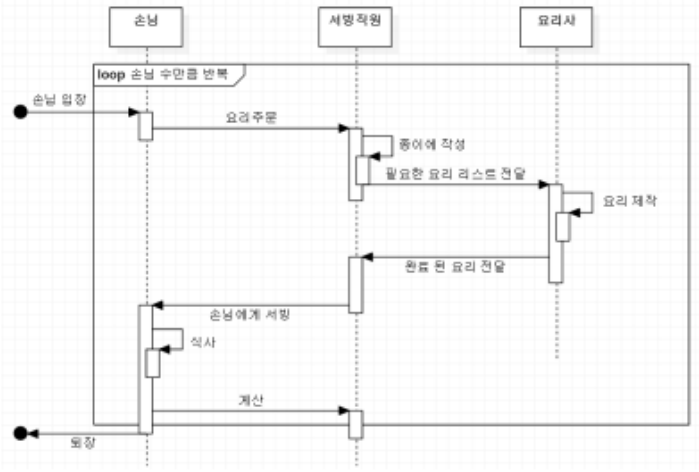
▶ 모델링과 UML

✓ 클래스 다이어그램

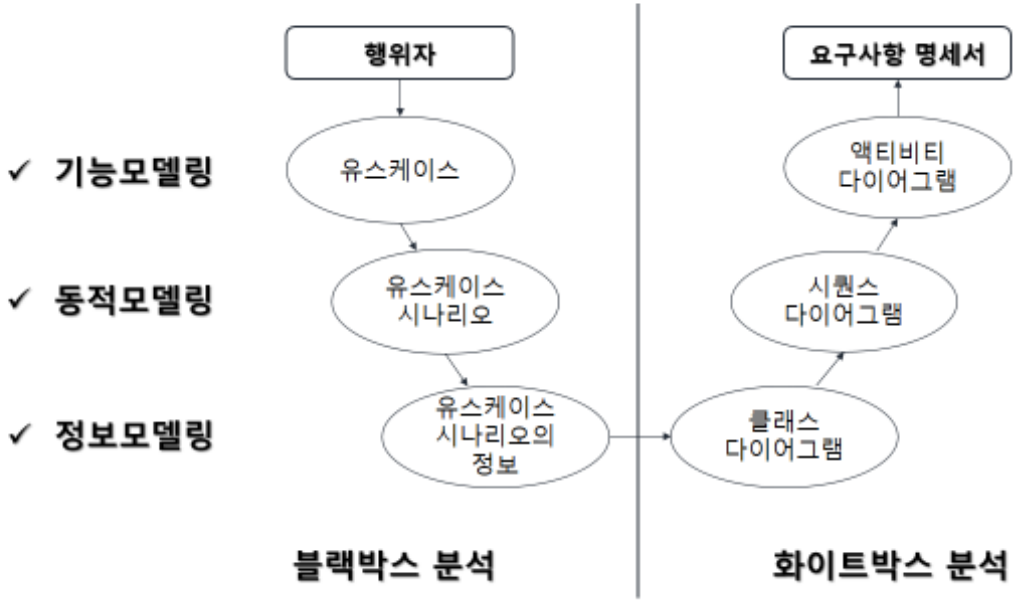


▶ 모델링과 UML

✓ 시퀀스 다이어그램



▶ UML의 V 프로세스



* **블랙박스 분석 :** 말 그대로 블랙박스의 관점으로 시스템을 분석하는 것이다. 즉, 사용자의 입장에서 분석하는 내용이 블랙박스 분석

* **화이트박스 분석 :** 사용자의 관점이 아닌 시스템의 관점으로 분석한 것이다.

* **유스케이스 시나리오가 포함할 것 :** 유스케이스를 수행하는 행위자, 선행 및 후행조건, 유스케이스의 이벤트 흐름

*** 분석 모델링 순서**

1. 요구사항 기술서 → 액터, 유스케이스 추출
2. 액터와 유스케이스 사이 관계 설정 → 유스케이스 다이어그램
3. 유스케이스 시나리오
4. 시나리오 → 속성, 관계 도출 → 클래스 다이어그램
5. 시나리오 → 오퍼레이션 도출 → 시퀀스 다이어그램
6. 4번의 클래스 다이어그램에 5번의 오퍼레이션 반영 → 분석 클래스 모델 완성
7. 액티비티 다이어그램 (시퀀스에서 표현 못한 복잡한 제어흐름 표현)



▶ UML 툴 활용

✓ StarUML 5.0 활용

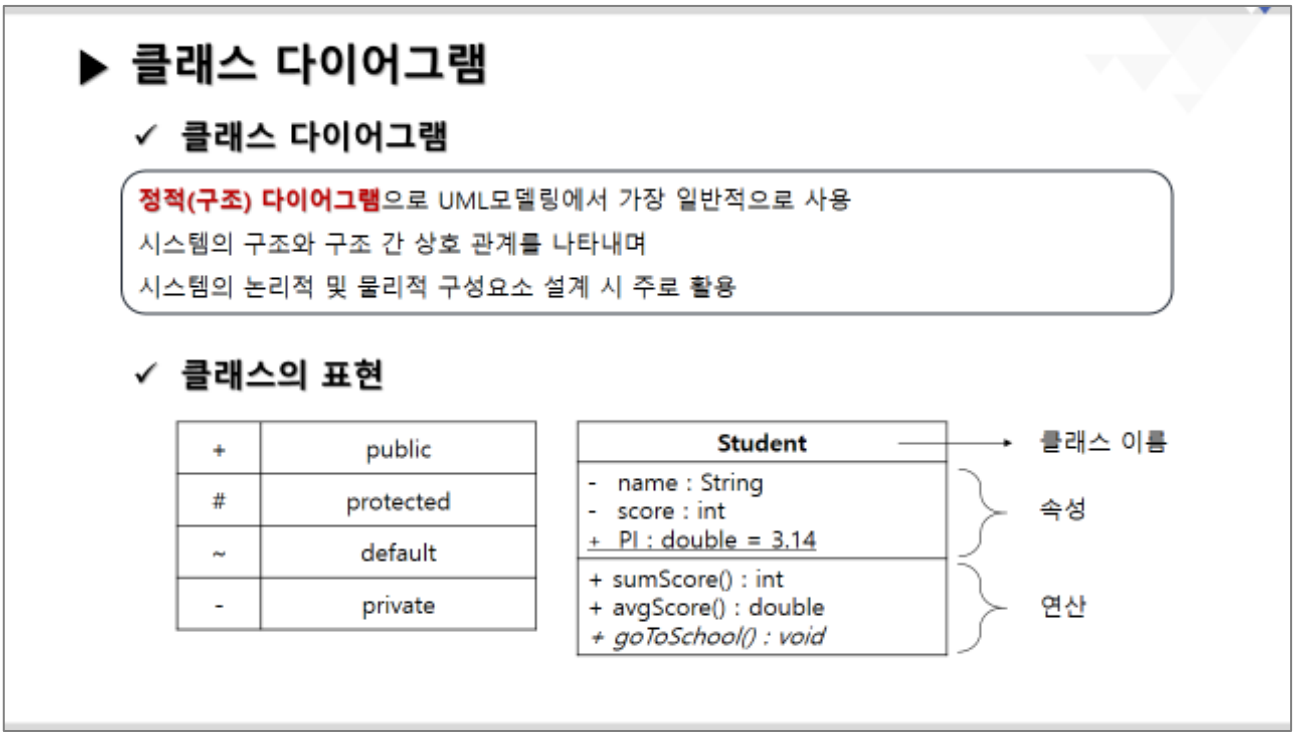
1. New Project By Approach
   Empty Project – Ok

2. 우측 Untitled 우클릭 – Add - Model
   (Default로 Model1이라는 Model이 만들어짐)
3. Model1 우클릭 – Add Diagram 후
   원하는 다이어그램 선택

4. 좌측에 생기는 기호를 끌어다가 쓰면서 다이어그램 완성

Chapter 8. 요구 기능별 애플리케이션 설계

클래스 다이어그램



*** 표기법**

	Uml	Java	C++
유사객체들	클래스		
상태	속성	필드	데이터멤버
행동	연산	메소드	멤버함수

*** 유도 속성 부가 설명**

- 다른 속성에 의해서 결정 될 수 있는 속성

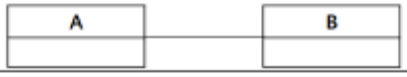
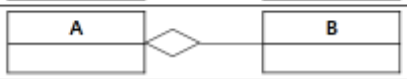
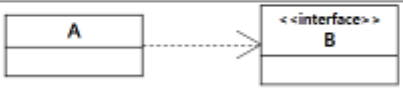
→ 자신의 다른 속성 뿐만 아니라 다른 클래스와의 관계로부터 결정될 수도 있음

- 표기법 : 가시성과 속성 이름 사이에 / 붙이기 (ex. - / age : int)
- 유도 속성의 값이 어떻게 다른 속성으로부터 계산 될 수 있는지를 명시적으로 기술하는 것이 바람직

→ 이점 : 다시 계산하지 않고 유도 속성에 저장된 값을 반환하면 되므로 성능 상의 이점이 있을 수 있음(ex. 갱신 회수보다 조회 회수가 많을 때)

→ 유도 속성의 무결성(integrity)를 유지하는데 주의

▶ 클래스 다이어그램의 관계

관계	표기법	의미
연관 관계		클래스 A와 클래스 B는 연관 관계를 가지고 있다.
합성(포함)관계		클래스 A는 클래스 B를 한 개 이상 포함하고 있다.
집합 관계		클래스 B는 클래스 A의 부분이다.
일반화 관계		클래스 B는 클래스 A의 하위 클래스이다.
실체화 관계 (인터페이스 실현 관계)		클래스 B는 인터페이스 A를 실현한다.
의존 관계		클래스 A는 클래스 B에 의존한다.
인터페이스 의존 관계		클래스 A는 인터페이스 B에 의존한다.

*** 인스턴스의 표기법**

표기법	인스턴스의 개수
1	1개
0..1	0개 또는 1개
*	다수
1..*	1개 또는 다수

*** 스테레오 타입** : UML의 한정된 모델 요소를 가지고 새로운 어휘를 표현하기 위한 방법

<<constructor>>, <<interface>>, <<abstract>> 등등 표시할 수 있음

▶ 연관 관계

✓ 연관 관계

한 클래스가 다른 클래스를 필드로 가지면서 클래스 간의 관련성을 뜻하는 것으로
메시지 전달의 통로 역할을 함

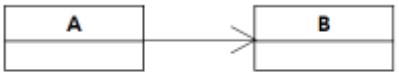



```
public class A{
        private B b;
}
```

```
public class B{
        private A a;
}
```

✓ 방향성이 있는 연관 관계

방향성은 메시지 전달의 방향을 뜻하며 반대 방향은 불가능



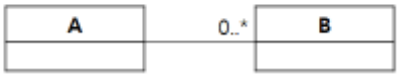

```
public class A{
        private B b;
}
```

```
public class B{
        private A a;
}
```

▶ 연관 관계

✓ 연관 관계의 다중성

관계를 맺을 수 있는 실제 상대 객체의 수를 다중성을 통하여 지정 가능
동일한 의미/역할의 복수 개의 객체와의 관계

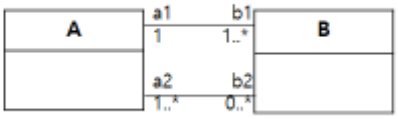



```
public class A{
    private Collection<B> b;
}
```

```
public class B{
    private A a;
}
```

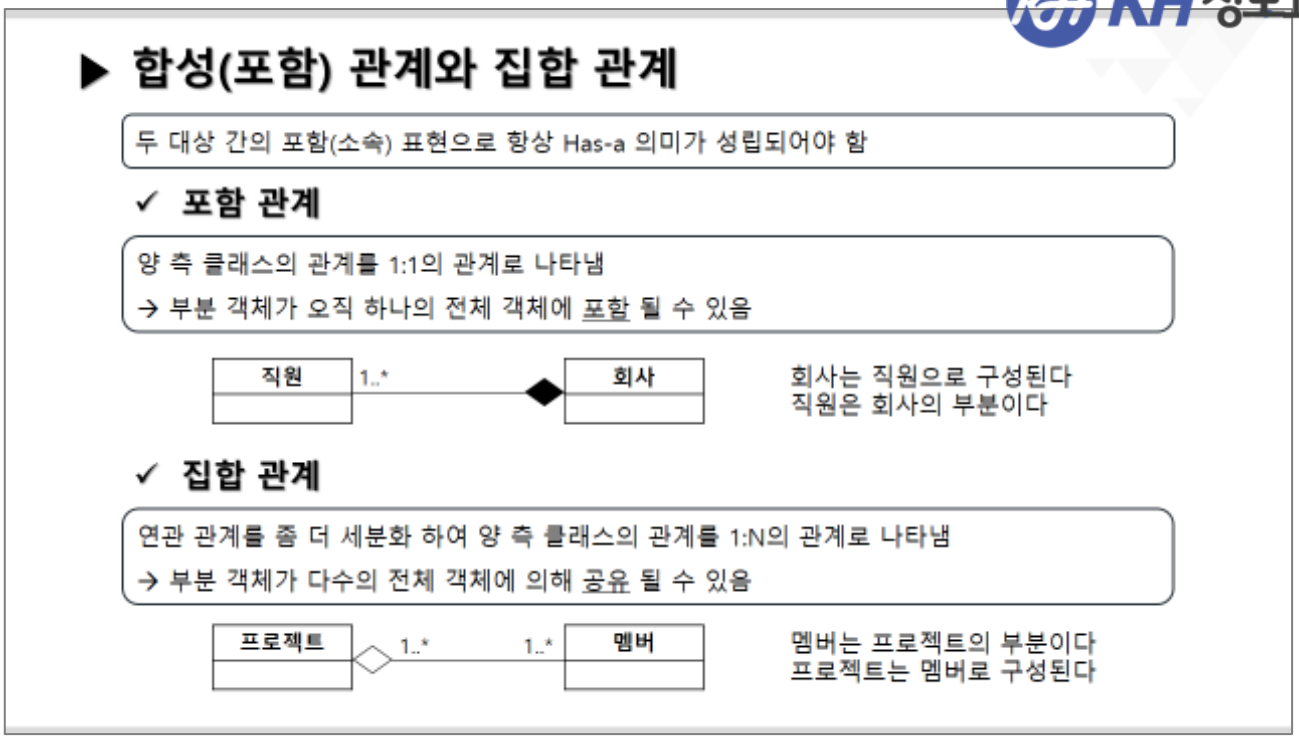
✓ 다중 연관

동일한 클래스 간의 존재하는 복수 개의 연관 관계를 뜻 함
다른 의미/역할의 복수 개의 객체와의 관계




```
public class A{
    private Collection<B> b1;
    private Collection<B> b2;
}
```

```
public class B{
    private A a1;
    private Collection<A> a2;
}
```



*** 공통점**

- **Has-a :** "전체는 부분으로 구성된다" , "부분은 전체의 부분이다"

*** 차이점**

- **집합 관계 :** 부분 객체가 다수의 전체 객체에 의해 공유 될 수 있다.

  ex. Project ◇— Member

  Project는 Member의 부분을 다른 Project에도 포함 시킬 수 있음

- **포함 관계 :** 부분 객체가 오직 하나의 전체 객체에 포함 될 수 있다.

  ex. Project ◆— Team

  ◇

  |

  Member

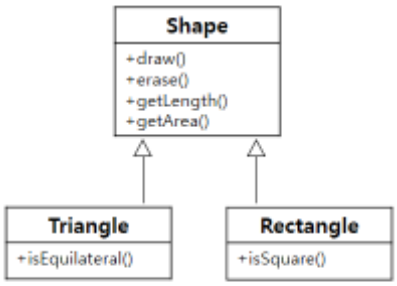
  Team의 부분은 다른 Project에 포함 될 수 없고

  Member의 부분은 다른 Team에 포함 될 수 있다.

▶ 일반화 관계와 실체화(인터페이스 실현) 관계

✓ 일반화 관계

보다 일반적인 클래스와 보다 구체적인 클래스 간의 관계를 뜻하는 것으로
한 클래스(상위 클래스)가 다른 클래스(하위 클래스)보다 일반적인 개념/대상 임을 의미하는 관계




```
public class Shape {
        public void draw() {...}
        public void erase() {...}
        public int getLength() {...}
        public double getArea() {...}
}

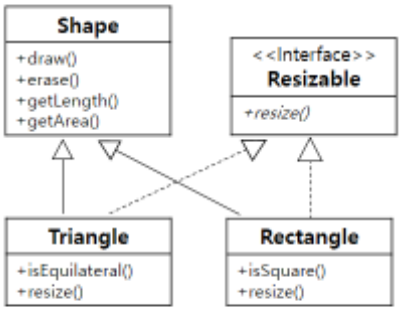
public class Triangle extends Shape {
        public boolean isEquilateral() {...}
}

public class Rectangle extends Shape {
        public boolean isSquare() {...}
}
```

▶ 일반화 관계와 실체화(인터페이스 실현) 관계

✓ 실체화(인터페이스 실현) 관계

인터페이스에 명세 된 기능을 클래스에 의해서 구현한 관계 의미




```
public interface Resizable {
        void resize();
}

public class Triangle extends Shape
                implements Resizable  {
        public boolean isEquilateral() {...}
        public void resize() {...}
}

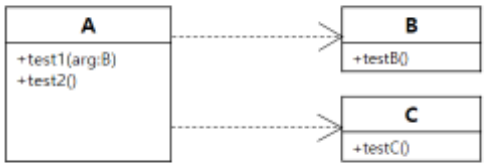
public class Rectangle extends Shape
                implements Resizable {
        public boolean isSquare() {...}
        public void resize() {...}
}
```

▶ 의존 관계와 인터페이스 의존 관계

✓ 의존 관계

두 클래스의 연산 간의 호출 관계를 표현한 것으로 제공자의 변경이 이용자에 영향을 미칠 수 있음을 의미(제공자의 변경이 이용자의 변경 유발)

이용자는 의존 관계를 통해서 제공자의 연산을 호출할 수 있음

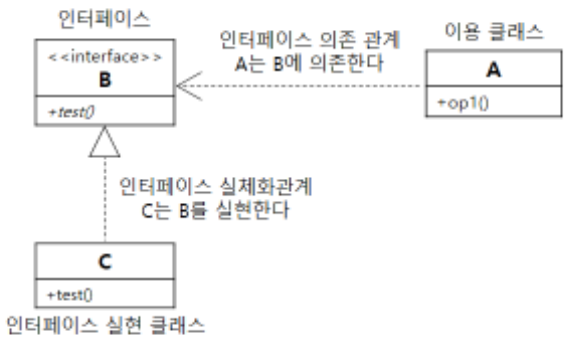



```
public class A{
        public void test1(B arg){
                arg.testB();
        }
        public void test2(){
                C c = new C ();
                c.testC();
        }
}
```

▶ 의존 관계와 인터페이스 의존 관계

✓ 인터페이스 의존 관계

인터페이스와 인터페이스 이용자 간의 이용관계를 표현할 때 사용 될 수 있음




```
public interface B{
        void test();
}
public class C implements B{
        public void test() {...}
}
public class A{
        public void op1(){
                B b = new C();
                b.test();
        }
}
```

▶ 연관 관계와 의존 관계

	연관 관계	의존 관계
역할	메시지 전달의 통로	
관계의 발생 형태	상대 클래스의 속성으로 대응	해당 연산의 인자 클래스 해당 연산 내부 객체의 클래스
관계의 지속 범위	해당 객체의 생명주기	해당 연산 내부
방향성	양방향 가능	단방향

< 실습 >

1. 어제 작업했던 유스케이스 다이어그램을 통해 3~4개 정도의 시나리오 배포

1. 개요

인터넷 쇼핑몰에 회원으로 가입할 수 있는 시나리오이다. 고객은 인터넷 쇼핑몰로부터 상품 구매 등의 서비스를 이용하기 위해 회원으로 가입한다.

2. 관계
- 개시자 : 고객
- 지원자 :
- 사전 조건 : 고객이 회원가입 기능을 실행한다.
- 사후 조건 : 고객의 정보가 시스템에 저장되고, 쇼핑몰의 회원이 된다.

3. 기본 흐름
   1) 고객이 회원가입을 신청한다.
   2) 회원으로 가입하기 위해 필요한 정보(이름, 아이디, 비밀번호, 주소, 주민등록번호, 메일, 휴대폰 번호)를 입력한다.
   3) 입력한 아이디가 기존에 가입된 회원의 정보와 중복 되는지 여부를 확인한다. 기존의 회원 정보와 중복되지 않았다면 가입이 완료된다.

4. 대안 흐름

5. 예외 흐름
   E1. 아이디가 중복된 경우
      1) 아이디는 시스템이 사용자 개개인을 구별하기 위한 정보로서 중복이 허용되지 않기 때문에, 시스템은 이미 동일한 아이디가 사용되고 있다는 메시지를 보여주고 다른 아이디를 입력하도록 해당 필드로 커서를 이동한다.

2. 시나리오 상의 기본흐름을 통해 속성, 오퍼레이션, 관계 도출

3. 클래스 다이어그램 만들기

*** 모범 답안 및 세부 내용은 별로 자료를 참고해주세요**

Day4. 시퀀스 다이어그램

▶ 시퀀스 다이어그램

✓ 시퀀스 다이어그램

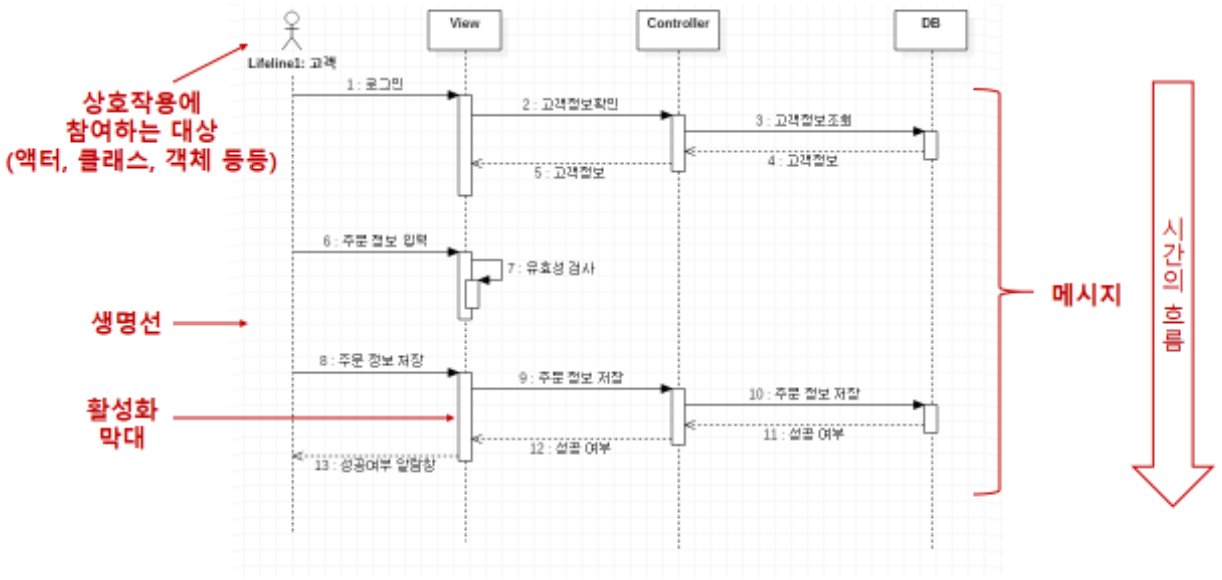
동적(행위) 다이어그램으로 상호작용 다이어그램의 일부분
시스템 내부에서 동작하는 객체들 사이의 주고 받는 메시지를 시간 순서를 강조하여 표현

✓ 생명선과 메시지

생명선 : 액터, 클래스 객체, 컴포넌트의 인스턴스 등, 상호작용에 참여하는 구체적인 대상 표현
생명선 끝에 X자로 끊겨 있으면 소멸
메시지 : 생명선 간에 전달되어 상태의 행위에 대한 호출

* 생명선의 소멸은 자바에서는 가비지 컬렉터에 넘기는 행동이 됨

▶ 시퀀스 다이어그램

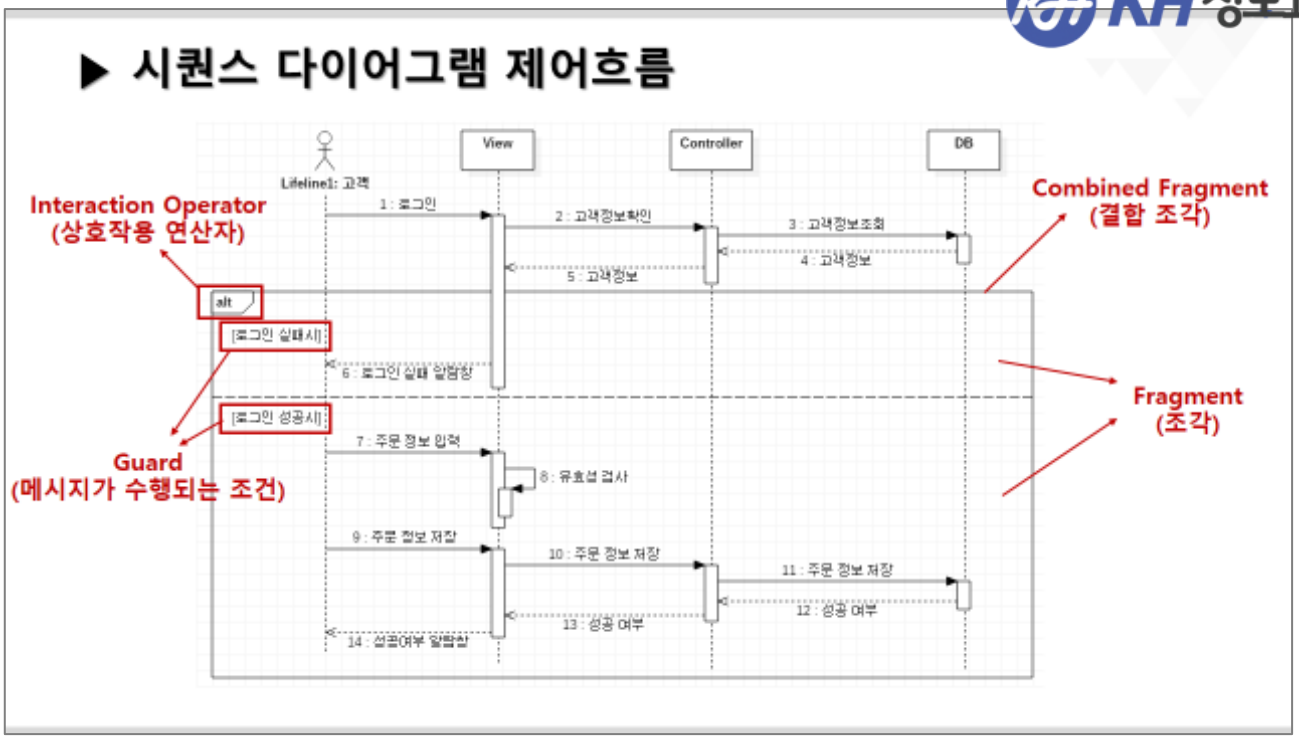


▶ 시퀀스 다이어그램 메시지 종류

메시지 유형	표현법	설명
비동기적 메시지		송신자를 대기 시키지 않음
동기적 메시지		송신자를 대기시킴
대답 메시지		동기적 메시지의 수행 결과
생성 메시지		생명선 생성
발견된 메시지		모르는 송신자로부터의 메시지
유실된 메시지		모르는 수신자로부터의 메시지

* **비동기 메시지 :** 메시지를 보내는 객체가 메시지를 보낸 후 즉시 제어를 받고, 메시지를 받는 객체가 다른 스레드의 제어 흐름에서 실행 될 수 있음

* **동기 메시지 :** 메시지를 받은 객체가 수행을 마칠 때 까지 제어를 되찾지 못하는 경우



*** 좋은 예**

- 시퀀스다이어그램 : 기본흐름

- 액티비티다이어그램 : 대체흐름 및 예외흐름

하지만 시퀀스로도 당연히 표현 가능!!

우리는 액티비티다이어그램을 다루지 않기 때문에 시퀀스에서 표현해주도록 하자!!!

시퀀스 다이어그램에서 결합조각(Combined Fragment)라는 걸 이용하여 제어 구조를 처리할 수 있다. 즉 반복, 분기, 조건 등등에 해당 되는 처리를 정의할 수 있다.

먼저 결합조각에서의 상호작용 연산자를 보면 alt라고 써있는데 이 밖에도 loop, opt, break 등등 열두개 정도의 연산자를 가지고 있다.

이 연산자를 통해 내가 어떠한 제어구조를 가질지 선택할 수 있는데 이러한 종류들은 다음페이지에서 다양하게 설명하도록 하고 여기서의 alt는 if-else문과 같은 논리라고 생각하면 된다.

그러면 우리는 어떠한 조건을 작성해야되지 않냐 → 바로 이 부분이 Guard(가드)라는 부분이다. 이는 상호작용의 피연산자라고 할 수 있으며 한 조건(가드)마다 실행되는 부분이 존재한다. 이를 조각이라고 한다. 따라서 가드의 조건이 true일 경우 해당 조각이 실행된다.

이러한 조각들이 결합되었다고 해서 결합조각이라 한다.

▶ 시퀀스 다이어그램 상호작용 연산자 종류

연산자 종류		설명
alt	대체	메시지의 대체 시퀀스를 포함하는 연산자이며 어떤 상황에서도 하나의 시퀀스만 발생함 → if - else if 문에 해당하는 논리를 나타냄. 단, 모든 가드가 false이고 else가드가 존재하지 않으면 어떠한 조각도 실행되지 않음
opt	옵션	선택적 요소로 발생하거나 발생하지 않을 수 있는 시퀀스를 포함 → if 문에 해당하는 논리를 나타냄. 대안을 하나만 제공해야 하고 조건이 false일 경우 조각이 실행 되지 않음
loop	반복	루프 상호작용 연산자는 반복적으로 실행되는 부분을 나타냄 가드 안에 단편이 실행되는 횟수를 지정할 수 있음
break	중단	보통 loop 연산자와 함께 쓰이며 중단 상호작용 연산자는 기타 프로그래밍 언어의 중단 매커니즘과 유사함 조건이 true 일 경우 현재 실행을 포기하고 빠져나감

* 이 외에 critical, ignore, strict, seq 등이 존재한다.

< 실습 >

1. 클래스 다이어그램 실습 때 배포했던 시나리오들 가져오기

1. 개요

인터넷 쇼핑몰에 회원으로 가입할 수 있는 시나리오이다. 고객은 인터넷 쇼핑몰로부터 상품 구대 등의 서비스를 이용하기 위해 회원으로 가입한다.

2. 관계
- 개시자 : 고객
- 지원자 :
- 사전 조건 : 고객이 회원가입 기능을 실행한다.
- 사후 조건 : 고객의 정보가 시스템에 저장되고, 쇼핑몰의 회원이 된다.

3. 기본 흐름
   1) 고객이 회원가입을 신청한다.
   2) 회원으로 가입하기 위해 필요한 정보(이름, 아이디, 비밀번호, 주소, 주민등록번호, 메일, 휴대폰 번호)를 입력한다.
   3) 입력한 아이디가 기존에 가입된 회원의 정보와 중복 되는지 여부를 확인한다. 기존의 회원 정보와 중복되지 않았다면 가입이 완료된다.

4. 대안 흐름

5. 예외 흐름
   E1. 아이디가 중복된 경우
      1) 아이디는 시스템이 사용자 개개인을 구별하기 위한 정보로서 중복이 허용되지 않기 때문에, 시스템은 이미 동일한 아이디가 사용되고 있다는 메시지를 보여주고 다른 아이디를 입력하도록 해당 필드로 커서를 이동한다.

2. 시나리오 상의 기본흐름만을 가지고 시퀀스 다이어그램 작성

3. 시나리오에 기본흐름 외의 대안, 예외 흐름에서도 속성, 오퍼레이션 도출

4. 2번에서 작성했던 시퀀스 다이어그램에 대안, 예외흐름을 추가적으로 작업해보기

*** 모범 답안 및 세부 내용은 별로 자료를 참고해주세요**